A LOW-MEMORY SPECTRAL-CORRELATION ANALYZER FOR DIGITAL QAM-SRRC WAVEFORMS

DYLAN JACOB GORMLEY

Bachelor of Science in Computer Engineering

University of Maryland, Baltimore County

May 2017

submitted in partial fulfillment of requirements for the degree

MASTER OF SCIENCE IN ELECTRICAL ENGINEERING

at the

CLEVELAND STATE UNIVERSITY

MAY 2021

We hereby approve this thesis for

DYLAN JACOB GORMLEY

Candidate for the Master of Science in Electrical Engineering degree for the

Department of Electrical Engineering and Computer Science

and the CLEVELAND STATE UNIVERSITY'S

College of Graduate Studies by

Professor Chansu Yu — Committee Chairperson

Electrical Engineering and Computer Science — 05/07/2021
Department — Date

Associate Professor Pong P. Chu — Committee Member

Electrical Engineering and Computer Science — 05/07/2021
Department — Date

Associate Professor Murad Hizlan — Committee Member

Electrical Engineering and Computer Science — 05/07/2021
Department — Date

05/07/2021

Student's Date of Defense

DEDICATION

To my dad, for his unconditional love and support.

ACKNOWLEDGMENT

I thank the following people and organizations:

My adviser, Professor Murad Hizlan, for teaching me communications and signal processing theory and for guiding every step of my journey with flexibility, patience, and encouragement. Signal detection and estimation theory has changed the way I view the world.

NASA Glenn Research Center for funding my master's degree. Working in the Cognitive Signal Processing Branch is a dream come true. Thanks to Gene and Dave and to my colleagues Joe, Mick, Adam, and Aaron, who have shaped me into the researcher I am today. This is the most cohesive, warm, and driven team I have ever worked with. They even helped me with my coursework—impeccable. I hope the R119 crew has many years ahead. Time to put this thing in space.

Dr. Chad M. Spooner for his feedback on the theory in this thesis. He held me accountable for every equation, plot, and concept. Without his blog on cyclostationary signal processing, I doubt this thesis would have been possible.

Anthony A. Stock for independently verifying and validating the digital designs presented in this thesis. I have enjoyed watching him grow into a skilled engineer, and I am proud of his hard work. I wish him good luck with whatever comes next!

Erica K. Henrichsen for creating the block diagrams presented in this thesis and for helping me through life's day-to-day trials. Her skills as a graphic designer are remarkable. I thank her for believing in me, even when I did not believe in myself.

My dad, John; my stepmother, Barb; my mom, Amy; and my sister, Cailin, for putting up with me over the years—I love them all very much. My cousins Megan, Mandi, Jimmy, and Liz, for lifting me up during my darkest days, year after year. My friends David, Greg, Jake, and Kent, for reminding me that life can be fun—they are like brothers to me. Lastly, and most importantly, my cat, Copernicus, for keeping me company during the arduous process of writing this thesis.

A LOW-MEMORY SPECTRAL-CORRELATION ANALYZER FOR DIGITAL QAM-SRRC WAVEFORMS

DYLAN JACOB GORMLEY

ABSTRACT

Cyclostationary signal processing (CSP) reveals periodic statistical features in received waveforms without prior synchronization. This thesis presents an FPGA-targeted VHDL architecture for blind symbol-rate and center-frequency estimation of M-ary quadrature amplitude modulation (QAM) waveforms shaped by a square-root-raised-cosine (SRRC) pulse for resource-constrained CubeSat cognitive radios. The streaming spectral correlation analyzer processes samples one at a time: it mixes each candidate center frequency to baseband, applies a two-tap finite-impulse-response filter, estimates the zero-lag cyclic autocorrelation at a candidate symbol rate, and compares the magnitude-squared statistic with an empirically selected, power-scaled threshold. MATLAB models reproduce the expected cyclic features, while fixed-point VHDL simulations driven by a Python transmitter and channel model evaluate modulation order, roll-off, requested grid spacing, capture length, word length, noise, sample rate, and two-signal operation. All thirteen experiments produce at least one threshold crossing, and eleven identify the transmitted symbol-rate–center-frequency pair or pairs. Grid-limited Experiment VII selects the first available rate of 1.00 MBd, and Experiment X selects the nearest evaluated center frequency of 0.04 MHz, illustrating the configured search resolution. The sample-by-sample datapath avoids an FFT sample buffer. Serial search time scales with capture length and the number of candidate pairs, while frequency spacing follows the numerically controlled oscillator's control-word width. The RTL uses a nonnegative power format, deterministic record boundaries, appropriately sized and normalized accumulators, and a continuous-rate interface.

TABLE OF CONTENTS

Page

LIST OF TABLES

LIST OF FIGURES

Symbols

a_i	ith data symbol
$b[i]$	ith finite-impulse-response filter coefficient
BW	sample bit width
$\widehat{C}_x(\alpha_v, f_k)$	finite-record, channelized zero-lag cyclic-autocorrelation statistic for x
$\widehat{C}_y(\alpha_v, f_k)$	finite-record, channelized zero-lag cyclic-autocorrelation statistic for received data y
D	real-valued Gaussian random variable
D_{dec}	integer decimation factor
E_b/N_0	received energy per information bit divided by noise spectral density
$\mathcal{F}$	set of center-frequency candidates
F_s	sampling frequency (sample rate)
f	spectral frequency
f_c	transmitted center frequency
f_k	kth center-frequency candidate
f_0	selected physical frequency for Fourier analysis
f_{LO}	local-oscillator frequency
$h[n]$	discrete-time filter impulse response
k	center-frequency-candidate index
L	number of finite-impulse-response filter taps
M	modulation order
N	number of samples in one observation record
N_k	number of center-frequency candidates
N_v	number of cycle-frequency or symbol-rate candidates
n	zero-based discrete-time sample index
$p(t)$	pulse-shaping waveform
$p_D(d)$	probability density of the random variable D
P_{CR}	probability of correct rejection
P_{D}	probability of detection
P_{FA}	probability of false alarm
P_{M}	probability of missed detection
$\widehat{P}_y$	estimated average received power
q	integer numerically controlled oscillator control word

Symbols (continued)

$\widehat{Q}_y(\alpha_v, f_k)$	generic finite-record cyclic-feature estimate
$\mathcal{R}$	set of symbol-rate candidates
R_s	symbol rate
$R_X(\tau)$	autocorrelation function of the random process $X(t)$
$R_X^\alpha(\tau)$	cyclic autocorrelation function at cycle frequency α
r	square-root-raised-cosine roll-off factor
$S_X(f)$	power spectral density of $X(t)$
$S_X^\alpha(f)$	cyclic spectral density of $X(t)$ at cycle frequency α
T	observation or averaging duration
T_0	symbol period, $1/R_s$
$T_x(v,k)$	nonnegative detection statistic for x at candidate pair (v,k)
$T_y(v,k)$	nonnegative detection statistic for received data y at candidate pair (v,k)
t_0	symbol-timing offset
v	cycle-frequency or symbol-rate-candidate index
W	input word width
W_{acc}	accumulator word width
$X(t)$	continuous-time random process
$x(t)$	sample path of $X(t)$
$X[n]$, $x[n]$	discrete-time process and sample sequence, respectively
$z_k[n]$	filtered, center-frequency-shifted sequence for candidate f_k
Δf	spectral-smoothing or analysis-filter bandwidth
α	cycle frequency
α_v	vth physical cycle-frequency candidate
β	dimensionless empirical threshold scale factor
γ	decision threshold
ℓ	zero-based discrete-Fourier-transform bin index
μ, σ^2	mean and variance, respectively
σ_a^2	data-symbol variance
τ	time lag
θ_0	carrier-phase offset
ν	integer cycle-frequency harmonic index

Abbreviations

AF	autocorrelation function
AGC	automatic gain control
AWGN	additive white Gaussian noise
AXI4-Stream	Advanced eXtensible Interface 4 Stream protocol
BPSK	binary phase-shift keying
BRAM	block random-access memory
CAC	cyclic autocorrelation
CAC-FFT	cyclic-autocorrelation fast Fourier transform
CAF	cyclic autocorrelation function
CAF-DFT	cyclic-autocorrelation-function discrete Fourier transform
CF	center frequency
CFD	cyclostationary feature detector
CFE	center-frequency estimator
CR	cognitive radio
CSD	cyclic spectral density
CSP	cyclostationary signal processing
CPU	central processing unit
CubeSat	standardized nanosatellite form factor based on cubic units
DFT	discrete Fourier transform
DPSK	differential phase-shift keying
DSP	digital signal processing
DSSS	direct-sequence spread spectrum
ESD	energy spectral density
FAM	FFT accumulation method
FCW	frequency control word
FF	flip-flop
FFT	fast Fourier transform
FIR	finite impulse response
FPGA	field-programmable gate array
FSK	frequency-shift keying
FSM	frequency-smoothing method
GPU	graphics processing unit
IID	independent and identically distributed
I/O	input/output
IQ	in-phase and quadrature
LO	local oscillator

Abbreviations (continued)

LPF	low-pass filter
LSB	least significant bit
LUT	lookup table
LUTRAM	lookup-table random-access memory
MATLAB	matrix laboratory
MSB	most significant bit
MSK	minimum-shift keying
NCO	numerically controlled oscillator
OOK	on-off keying
PDF	probability density function
PSD	power spectral density
PSK	phase-shift keying
PU	primary user
QAM	quadrature amplitude modulation
RAM	random-access memory
RF	radio frequency
RTL	register-transfer level
SCA	spectral correlation analyzer
SNR	signal-to-noise ratio
SOC	second-order cyclostationary
SOS	second-order stationary
SR	symbol rate
SRE	symbol-rate estimator
SRRC	square-root-raised-cosine
SSCA	strip spectral correlation analyzer
SU	secondary user
SWaP	size, weight, and power
TSM	time-smoothing method
VHDL	very-high-speed integrated-circuit hardware description language
WSC	wide-sense cyclostationary
WSS	wide-sense stationary

CHAPTER I

INTRODUCTION

Many digitally modulated radio-frequency (RF) communication signals exhibit second-order statistical properties that vary periodically with time. This behavior is known as *cyclostationarity*. This thesis focuses on estimating the cycle frequencies of second-order cyclostationary (SOC) signals. In this work, we perform temporal analysis using the cyclic autocorrelation function (CAF) and spectral analysis using the cyclic spectral density function (CSD) [7].

Some standard CSD estimation techniques are the frequency-smoothing method (FSM), time-smoothing method (TSM), strip spectral correlation analyzer (SSCA), and fast Fourier transform (FFT) accumulation method (FAM) [7]. A realization of a CSD estimator is called a *spectral correlation analyzer* (SCA) [3, 7]. This thesis investigates SCA techniques and evaluates their suitability for severely resource-constrained platforms such as CubeSats.

Terrestrial receivers can often accommodate less stringent size, weight, and power (SWaP) constraints than spacecraft receivers. Higher-SWaP devices such as multicore central processing units (CPUs), graphics processing units (GPUs), and large-cell-count field-programmable gate arrays (FPGAs) can therefore be deployed to perform complex digital signal processing (DSP) routines.

For CubeSat cognitive radios (CRs), however, onboard resources are constrained. A candidate CR may need to collect data, process signals, receive commands, report

status, and store data while meeting spacecraft SWaP limits. Resource-conscious devices such as smaller FPGAs are therefore relevant implementation platforms.

Many established SCA algorithms rely extensively on FFTs, whose data buffering can consume substantial block random access memory (BRAM) in FPGA implementations. This thesis studies a time-domain SCA design that relies minimally on BRAM. The low-BRAM design operates on a sample-by-sample ("streaming") basis and avoids an FFT sample buffer, making the architecture suitable for resource-constrained CubeSat CR research and FPGA prototyping.

Chapter II begins by reviewing the basic notation and concepts of stochastic processes, also known as *random signals*. We then develop the intuition and mathematics behind cyclostationary signal processing (CSP). Next, we discuss the challenges associated with calculating the power spectral density (PSD) of random signals. PSD estimation techniques are discussed, followed by CSD estimation techniques for cyclostationary signals. We then evaluate several SCAs and discuss their suitability for our low-BRAM objective. Finally, we conclude with an overview of a technique that leverages CSP to intelligently reuse spectral bands, known as *spectrum sensing*.

In Chapter III, we first discuss the motivation for performing spectrum sensing in space. Next, we discuss a set of assumptions necessary for the design of our low-BRAM SCA. In MATLAB, we model a CAF-discrete Fourier transform (CAF-DFT) estimator that adapts a previously studied symbol-rate detector and center-frequency sweep to a serial hardware architecture. We demonstrate it using M-ary quadrature amplitude modulation (M-QAM) with square-root-raised-cosine (SRRC) pulse shaping, abbreviated M-QAM-SRRC. With a resource-constrained target in mind, we simplify parts of the design using a single-point estimator with little BRAM. We conclude this chapter with a comprehensive look at the digital architecture for this design, which we refer to as the *streaming SCA* because of its BRAM-reducing, stream-based approach.

In Chapter IV, we characterize our streaming SCA. The experiments stress-test various parameters, including bit widths, symbol rates, center frequencies, capture lengths, noise, and SRRC roll-off factors. We conclude this thesis in Chapter V with an overall summary and a discussion of opportunities for future work.

CHAPTER II

BACKGROUND

In this chapter, we review the concept of a random signal. We then briefly discuss the autocorrelation function (AF), followed by the CAF. After this, we describe the challenges of PSD and CSD estimation. Finally, we conclude with an overview of a technique that leverages CSP to reuse free spectral bands, known as *spectrum sensing*.

2.1 Cyclostationary Signal Processing

In this section, we summarize random signal notation and concepts as described in W. A. Gardner's book *Introduction to Random Processes with Applications to Signals and Systems* [2]. We begin with the AF and then build on this groundwork to introduce the concept of the CAF.

2.1.1 The Autocorrelation Function

We begin with a review of stochastic processes. For digital communications, these are typically referred to as *random signals*. A random signal $X(t)$ is a collection of random variables indexed by time. One outcome of the underlying random experiment produces a sample path, or time series, denoted by $x(t)$. Communication signals are often represented by complex-valued random processes, whereas the familiar one-dimensional Gaussian density describes a real-valued random variable $D \sim \mathcal{N}(\mu, \sigma^2)$ as follows:

$$p_D(d) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{d-\mu}{\sigma}\right)^2}. \tag{1}$$

The Gaussian PDF has the shape shown in Figure 1.

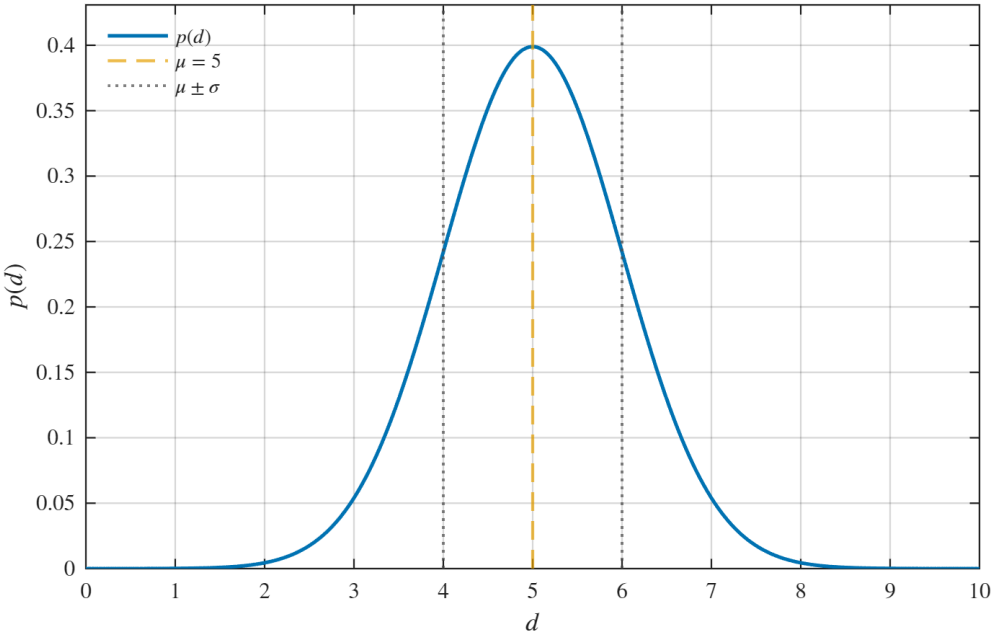

Figure 1: Example of a Gaussian PDF. Here, Equation (1) is evaluated for 500 values of d, with a mean of $\mu = 5$ and a standard deviation of $\sigma = 1$.

A random process is completely specified by all of its finite-dimensional joint distributions, or by the corresponding joint PDFs when those densities exist. Its mean is an ensemble average over realizations at a fixed time. For a real-valued process with marginal PDF $p_{X(t)}(x)$, the mean is

$$\mu_X(t) = E\{X(t)\} = \int_{-\infty}^{\infty} x\, p_{X(t)}(x)\, dx. \tag{2}$$

For a complex-valued process, the expectation in Equation (2) is applied to the real and imaginary parts. The ensemble average is distinct from a time average along one sample path; the two are equal only under the relevant ergodicity conditions. The

first raw moment $E\{X(t)\}$ is the mean $\mu_X(t)$ by definition, without any Gaussian or symmetry assumption [2].

The second central moment $\sigma_X^2(t) = E\{|X(t) - \mu_X(t)|^2\}$ is the *variance* of $X(t)$. For a real-valued process, the magnitude squared reduces to the ordinary square. The nonnegative square root $\sigma_X(t)$ is the *standard deviation*, which describes the spread of $X(t)$ about its mean [2].

The second joint moment $R_X(t_1, t_2) = E\{X(t_1)X^*(t_2)\}$ is the *AF*. For WSS random signals, the AF is expressed as $R_X(\tau)$, where $\tau = t_1 - t_2$. The variable τ is the time separation, or *lag*, between the two samples [2].

If the joint distributions of $X(t)$ through order n are invariant under a common shift of all their time arguments, the process is said to be nth-order stationary. A process with this invariance at every order is *strict-sense stationary*. A finite-second-moment process is second-order stationary (SOS), or *wide-sense stationary* (WSS), when its mean is constant and its AF is invariant under a common time shift, so that the AF depends only on lag [2, 7].

Let us further develop $X(t)$ by assuming it is a WSS random signal. Its AF is

$$R_X(\tau) = E\left\{X\left(t + \frac{\tau}{2}\right)X^*\left(t - \frac{\tau}{2}\right)\right\}. \tag{3}$$

Thus, a WSS process has a constant mean and variance, and its AF is a function of τ rather than of absolute time. For a correlation-ergodic WSS process, $R_X(\tau)$ can also be obtained as the infinite-time average along almost any sample path; a finite-record version is an autocorrelation estimate [2]. Figure 2 illustrates a WSS process with constant mean and variance.

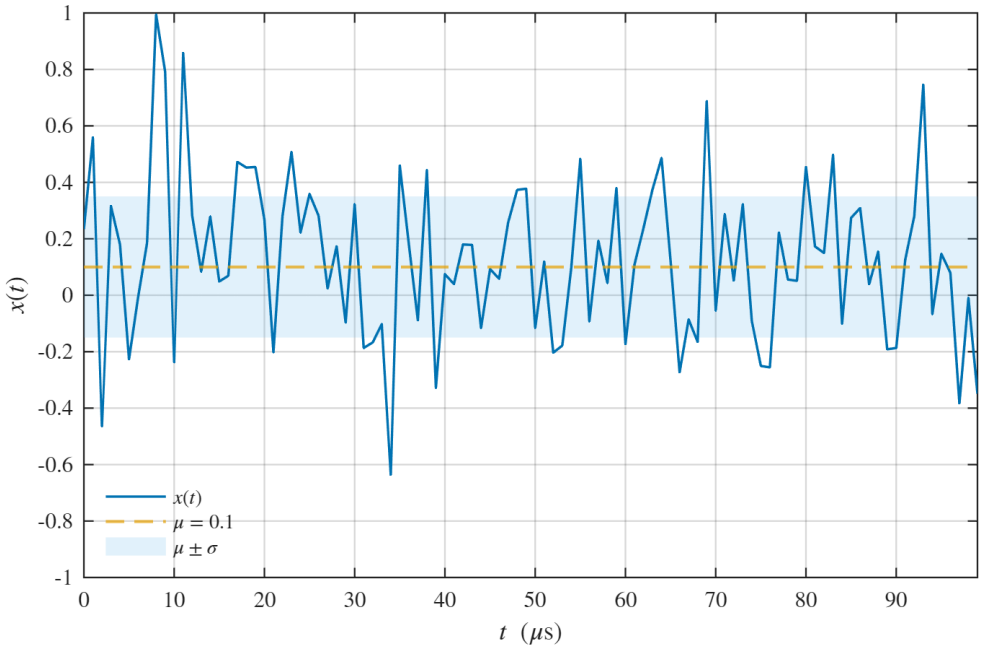

Figure 2: One 100-sample realization of an independent and identically distributed (IID) Gaussian process with constant mean $\mu_X = 0.1$ and standard deviation $\sigma_X = 0.25$. The dashed line is the theoretical mean and the shaded band is $\mu_X \pm \sigma_X$.

Many digitally modulated communication waveforms are not WSS but instead exhibit cyclostationarity. In this context, we use *waveform* to mean a modulated signal. Quadrature amplitude modulation (QAM) conveys information through the amplitudes of two orthogonal carriers at the same frequency, conventionally called the in-phase and quadrature components. Each symbol selects one of M constellation points, where M is the *modulation order*. Under the MATLAB convention used in this thesis, the $M = 2$ case reduces to two antipodal constellation points and is equivalent to binary phase-shift keying (BPSK). The rate at which symbols are transmitted is the *symbol rate* (SR), denoted R_s, and the modulated carrier frequency is the waveform's *center frequency* (CF), denoted f_c. An example of a 2-QAM time series is shown in Figure 3.

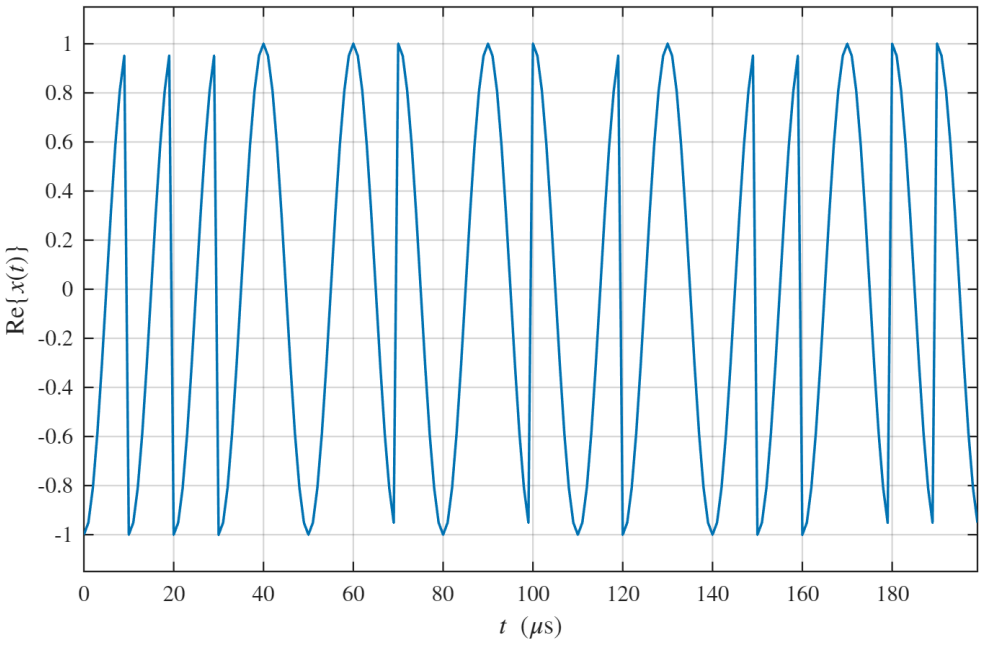

Figure 3: The real component of a 2-QAM waveform. The model generates 400 random symbols, applies a length-10 rectangular pulse, and displays the first 200 samples. The waveform parameters are $R_s = 0.1$ MBd, $f_c = 0.05$ MHz, and $F_s = 1$ MHz.

An M-QAM waveform has the following mathematical representation:

$$X(t) = \sum_{i=-\infty}^{\infty} a_i p(t - iT_0 - t_0) e^{j(2\pi f_c t + \theta_0)}, \tag{4}$$

where a_i is the ith data symbol, $p(t)$ is the pulse shape, $T_0 = 1/R_s$ is the symbol period, t_0 is the timing offset, f_c is the CF, and θ_0 is the carrier-phase offset. This linear-modulation model includes QAM and phase-shift keying (PSK) waveforms. Adaptive modulation and Nyquist pulse shaping are widely used communication techniques. This thesis narrows its investigation to the M-QAM-SRRC class of waveforms.

To conclude this section, Figure 4 plots $|\widehat{R}_x(\tau)|$ versus τ in what is known as an *autocorrelogram*.

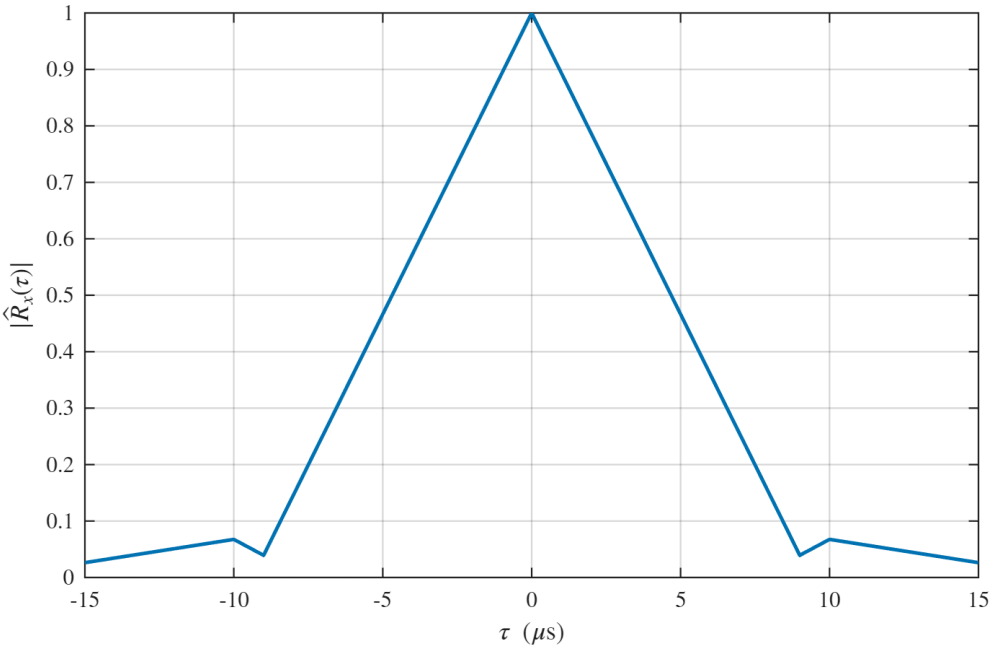

Figure 4: Magnitude of the biased autocorrelation estimate for the waveform $x(t)$ shown in Figure 3, evaluated at 31 integer-sample lags.

2.1.2 The Cyclic Autocorrelation Function

If the finite-dimensional distributions of $X(t)$ are periodic under a common shift of their time arguments, $X(t)$ is said to exhibit *cyclostationarity*. If its mean and AF are periodic under a common time shift, it is said to be *SOC* or *wide-sense cyclostationary* (WSC) [2, 7]. The ensemble CAF is the Fourier-series coefficient of the symmetric time-varying AF $R_X(t,\tau) = E\{X(t+\tau/2)X^*(t-\tau/2)\}$:

$$\widetilde{R}_X^{\alpha}(\tau) = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} R_X(t,\tau) e^{-j2\pi\alpha t} dt. \tag{5}$$

In Equation (5), α is the *cycle frequency*. More generally, the cyclic expectation $E^{\alpha}\{Z(t)\}$ denotes the α-frequency Fourier coefficient of the time-varying ensemble mean $E\{Z(t)\}$; it is not a different ordinary expectation. Under the appropriate correlation-cycloergodicity condition, the ensemble CAF equals the fraction-of-time

average computed from a single sample path [7]. The resulting sample-path expression is

$$R_X^{\alpha}(\tau) = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} x\left(t + \frac{\tau}{2}\right) x^*\left(t - \frac{\tau}{2}\right) e^{-j2\pi\alpha t} dt. \tag{6}$$

Figure 5 plots the magnitude of a finite-sample estimate of Equation (6) in what is known as the cyclic autocorrelogram.

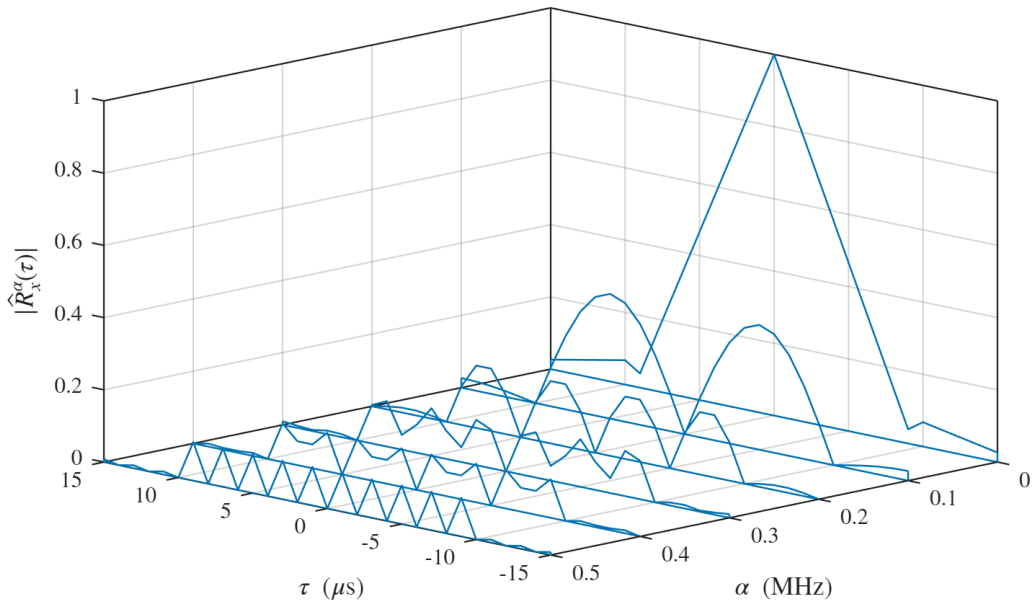

Figure 5: Magnitude of the biased cyclic-autocorrelation estimate for the waveform $x(t)$ shown in Figure 3, evaluated at 31 integer-sample lags and six cycle frequencies from 0 through 0.5 MHz.

Peaks in the finite-sample cyclic autocorrelogram indicate candidate cycle features of $X(t)$, providing information not present in the ordinary AF alone. We will build on this concept in the following three sections to explain the significance of cyclic features.

2.2 Spectral Density

This section continues the summary of notation and concepts described in [2]. We first review the periodogram, the challenges faced when estimating the PSD from the periodogram, and a PSD estimation method known as *frequency-smoothing*. We then build upon this work to develop the concept of the CSD.

2.2.1 The Power Spectral Density Function

A rectangular pulse is defined as

$$\Pi(t) = \begin{cases} 0, & if \quad |t| > \frac{1}{2} \\ 1, & if \quad |t| \leq \frac{1}{2} \end{cases} . \tag{7}$$

The Fourier transform of $\Pi(t)$ is denoted $\hat{\Pi}(f)$. Since $\Pi(t)$ is a finite-energy signal, its energy spectral density is defined by

$$\Psi_{\Pi}(f) = \left|\hat{\Pi}(f)\right|^2 . \tag{8}$$

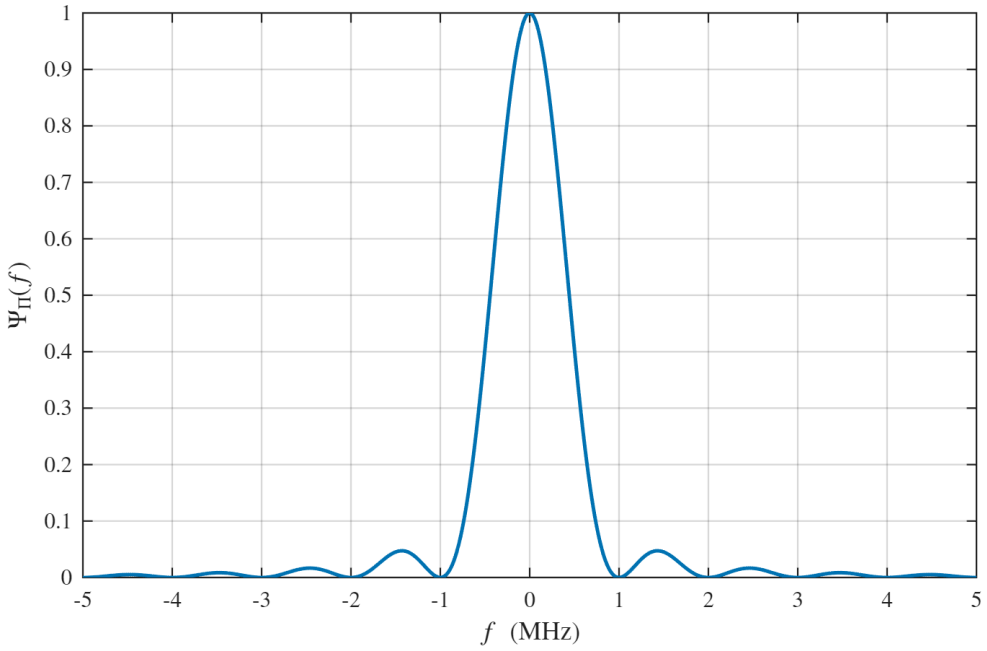

Figure 6: Squared-magnitude Fourier transform of the unit-width rectangular pulse $\Pi(t)$. At $F_s = 50$ MHz, the one-microsecond pulse occupies 50 samples within a 150-sample record; a zero-padded 4096-point frequency grid renders the curve smoothly, and the numerical ESD satisfies Parseval's energy equality.

Parseval's theorem states that the total spectral energy, $\int_{-\infty}^{\infty} \Psi_\Pi(f)\, df$, equals the time-domain energy, $\int_{-\infty}^{\infty} |\Pi(t)|^2\, dt$ [2].

A sustained QAM waveform is generally modeled as a power signal rather than a finite-energy signal, so an ordinary finite ESD is not the appropriate description. For a sample path $x(t)$ observed through a duration-T rectangular window, its periodogram is [2]

$$P_{x,T}(f) = \frac{1}{T} \left| \hat{x}_T(f) \right|^2, \tag{9}$$

where $\hat{x}_T(f)$ is the Fourier transform of the windowed record. The periodogram estimates the distribution of average power over frequency, but its variance generally does not vanish as T increases; it is therefore not, by itself, a consistent PSD estimator. Low signal-to-noise ratio further obscures signal features [2]. Figure 7 illustrates the

periodogram of the 2-QAM signal from Figure 3.

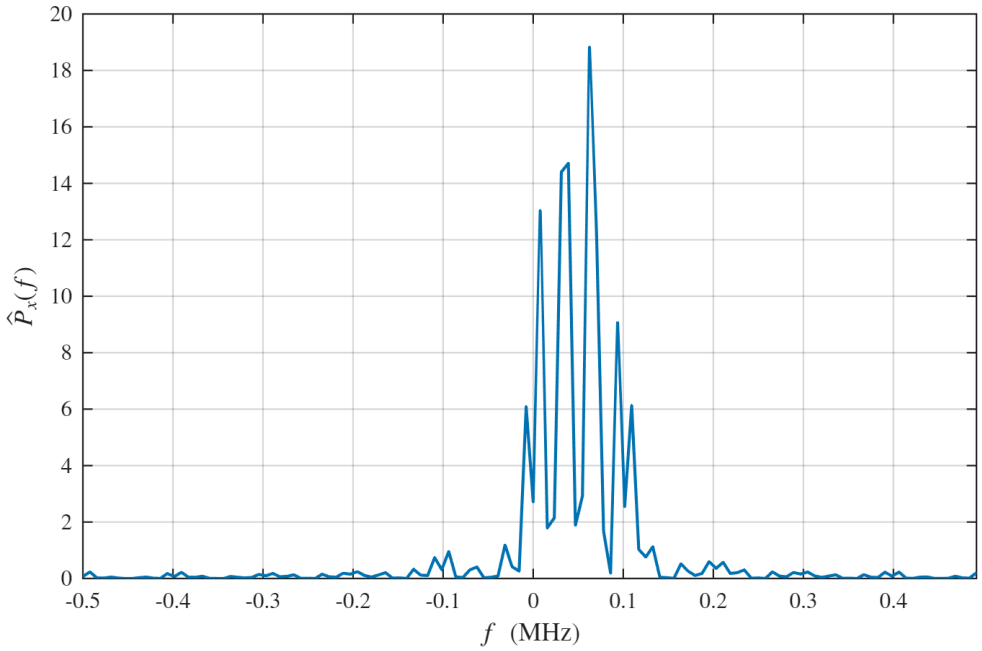

Figure 7: Squared-magnitude, power-scaled periodogram of the first 128 samples of the waveform $x(t)$ shown in Figure 3. The estimate uses a 128-point FFT.

The periodogram is roughly centered at f_c. Because this example uses rectangular pulses, its main lobe between the first spectral nulls has width approximately $2R_s$. Without prior knowledge, however, it would be challenging to infer the true CF and SR accurately from this one high-variance estimate.

One technique used to reduce this variability is the FSM [7]. Convolving the periodogram with a frequency-domain smoothing kernel $h(f)$ gives the estimate

$$\widehat{S}_X(f) = h(f) * P_{x,T}(f), \tag{10}$$

where $*$ is the convolution operator. Frequency smoothing reduces estimator variance at the cost of spectral resolution and can introduce bias. In Figure 8, it makes the CF easier to estimate, but excessive smoothing broadens spectral features and can degrade SR estimation. The smoothing bandwidth must therefore be chosen according to the

bias–variance and resolution tradeoffs; no single minimum amount is optimal for every record.

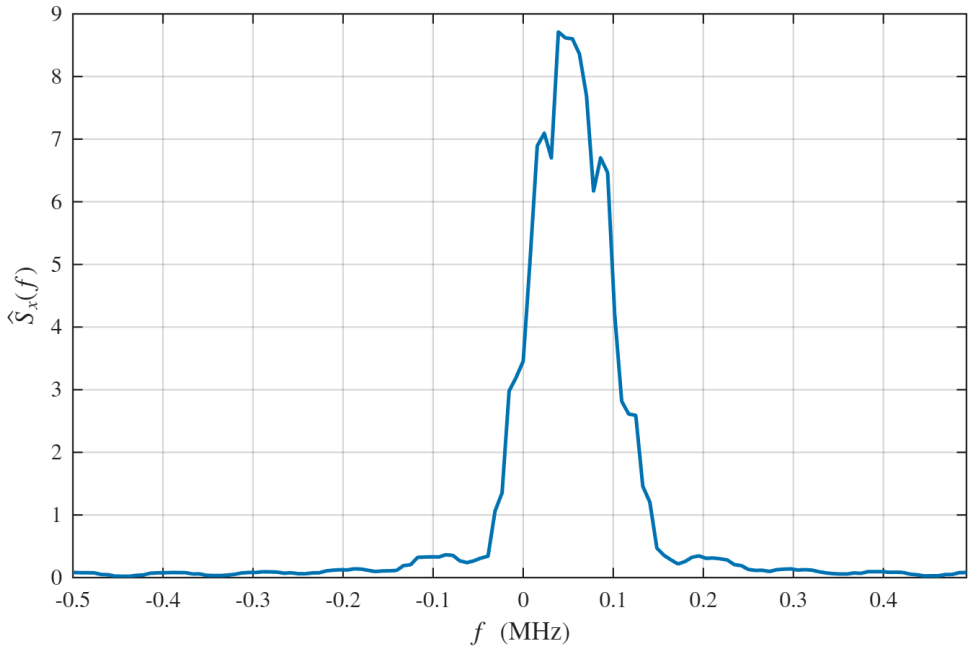

Figure 8: Frequency-smoothed PSD estimate of the waveform $x(t)$ shown in Figure 3. The 128-point periodogram is averaged over eight adjacent frequency bins.

2.2.2 The Cyclic Spectral Density Function

For a WSS random process, the theoretical PSD is the Fourier transform of $R_X(\tau)$. A finite-record periodogram estimates that PSD, and smoothing trades resolution for reduced variance. The analogous finite-record quantity for spectral correlation is the *cyclic periodogram*:

$$P^{\alpha}_{x,T}(f) = \frac{1}{T}\hat{x}_T\Big(f + \frac{\alpha}{2}\Big)\hat{x}^*_T\Big(f - \frac{\alpha}{2}\Big), \tag{11}$$

where $\hat{x}^*_T$ is the complex conjugate of $\hat{x}_T$. Its corresponding frequency-smoothed CSD estimate is

$$\widehat{S}_X^{\alpha}(f) = h(f) * P_{x,T}^{\alpha}(f). \tag{12}$$

Across all values of cycle frequency α and spectral frequency f, the theoretical CSD $S_X^{\alpha}(f)$ characterizes the second-order periodic correlation structure of $X(t)$; it does not, in general, specify the process's higher-order statistics or full probability law [2]. Equation (12) is a finite-record estimate of that quantity. The CSD estimate of $X(t)$ is shown in Figure 9.

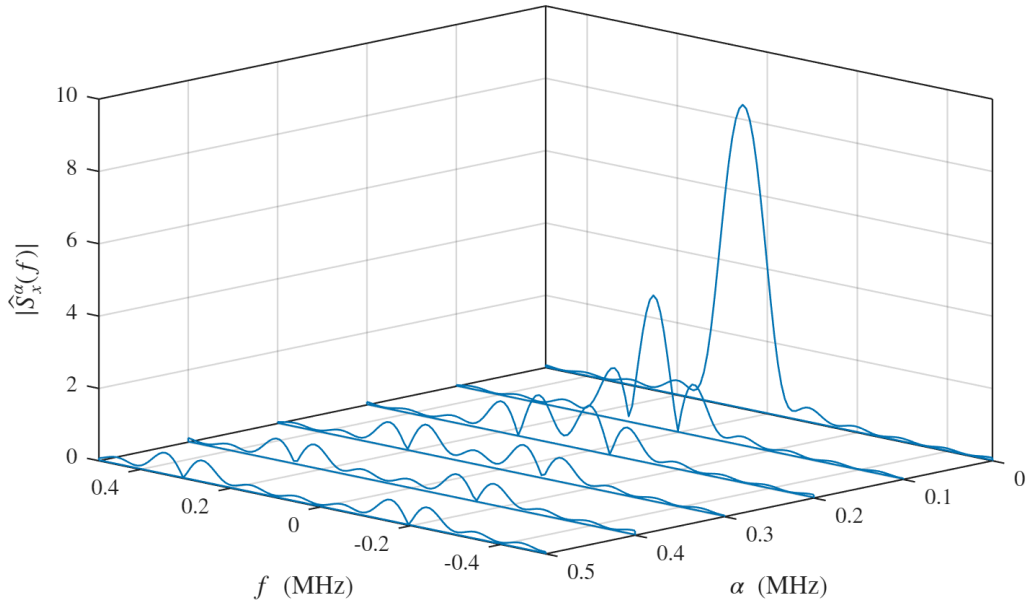

Figure 9: CSD magnitude estimate of the waveform $x(t)$ shown in Figure 3. A 128-point DFT is applied across the 31 estimated lags of $\widehat{R}_x^{\alpha}(\tau)$, followed by averaging over eight adjacent frequency bins.

At $\alpha = 0$, the CSD reduces to the ordinary, zero-cycle power spectrum. Components at $\alpha \neq 0$ reveal periodic second-order structure; ideal stationary noise has no such nonzero-cycle components.

2.3 Spectral Correlation Analyzers

In this section, we move on to realizations of the CSD. The process of producing the CSD of a received waveform with no prior information is called *blind estimation*. Realizations of a blind estimator are often called *SCAs*. First, we discuss an example of a TSM-SCA as described by W. A. Gardner in the article *Cyclostationarity: Half a Century of Research* [3]. This TSM-SCA is used to blindly estimate the CSD of arbitrary (α, f) pairs.

We then consider the case in which the cycle frequency of interest equals the SR. M. V. Koch's technical report *Interference Mitigation Using Cyclic Autocorrelation and Multi-Objective Optimization* describes a *CAC-FFT algorithm* that forms $|x[n]|^2$, uses an FFT to evaluate the zero-lag CAC on a uniform rate grid, and selects candidate symbol rates from its peaks [5]. For each candidate rate, the report's implemented center-frequency detector systematically shifts and low-pass filters the signal segment, then evaluates a single-rate CAC at each offset. Koch also reports preliminary work on a different center-frequency preprocessor, which was not used in the reported system. The estimator in this thesis adapts the implemented CAC and carrier-sweep operations to a serial, fixed-filter hardware architecture; we refer to its pointwise statistic as the *CAF-DFT*.

2.3.1 Blind Cycle- and Spectral-Frequency Estimation

The most basic realization of the time-smoothed CSD is the TSM-SCA. A finite time-smoothed CSD estimate can be written as

$$\widehat{S}_X^{\alpha}(t,f)_{T,\Delta f} = \frac{1}{T}\int_{t-T/2}^{t+T/2} \Delta f\, X_{1/\Delta f}\Big(u, f+\frac{\alpha}{2}\Big) X_{1/\Delta f}^{*}\Big(u, f-\frac{\alpha}{2}\Big)\, du. \qquad (13)$$

Here, $X_{1/\Delta f}(u,\nu)$ is the short-time Fourier transform centered at time u, with an effective duration of approximately $1/\Delta f$. The ideal CSD is approached by first letting $T \to \infty$ and then $\Delta f \to 0$; a finite estimate is statistically reliable only

when the time–bandwidth product $T\Delta f$ is sufficiently large. The TSM-SCA can be decomposed into multipliers, mixers, filters, and accumulators. To establish a baseline implementation, the block diagram based on Equation (13) is shown in Figure 10 [3].

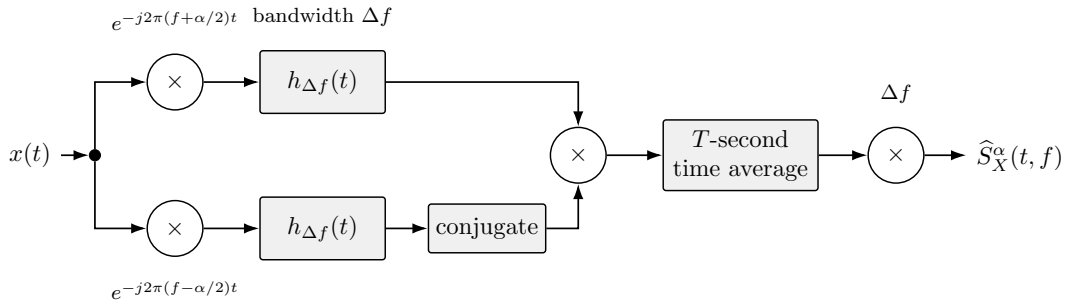

Figure 10: As described by Gardner in [3], this baseline SCA shifts the components centered at $f+\alpha/2$ and $f-\alpha/2$ to baseband. The component centers are separated by α. The shifted signals pass through low-pass filters $h_{\Delta f}(t)$ with bandwidth Δf and unity-passband height before their outputs are correlated, time-averaged over T, and scaled by Δf. The ideal CSD $S_X^\alpha(f)$ is obtained in the ordered limits $T \to \infty$ and then $\Delta f \to 0$.

Although the SCA shown in Figure 10 applies to an arbitrary waveform and gives the blind CSD estimate for any (α, f) pair, much of this search space is not helpful for most applications. If we can intelligently limit this search space, we can significantly improve the utility of the TSM-SCA.

2.3.2 Blind Symbol Rate-Center Frequency Point Estimation

Several common SCAs, such as the FSM, TSM, SSCA, and FAM, are discussed in [7]. Efficient digital implementations of several of these techniques use FFTs and block buffering, which can make them memory-intensive. Although larger memory allocations may be acceptable for terrestrial receivers, we must prioritize memory conservation for a CubeSat CR and explore alternative SCA designs.

Koch's CAC-FFT uses peaks in the DFT of $|x[n]|^2$ to estimate candidate symbol rates [5]. In the CAF-DFT used here, that operation is applied after channelization over candidate center frequencies. The relevant QAM-SRRC cycle-frequency struc-

ture follows from the linear-modulation model in Equation (4) [2]. Assume the data symbols are zero-mean and uncorrelated, with $E\{a_i a_m^*\} = \sigma_a^2 \delta_{im}$, and assume zero timing and carrier-phase offsets. The symmetric AF is then

$$R_X(t,\tau) = \sigma_a^2 e^{j2\pi f_c \tau} \sum_{i=-\infty}^{\infty} p\left(t + \frac{\tau}{2} - iT_0\right) p^*\left(t - \frac{\tau}{2} - iT_0\right). \tag{14}$$

Because Equation (14) is periodic with T_0, we can express the CAF as follows:

$$R_X^\alpha(\tau) = \frac{\sigma_a^2}{T_0} e^{j2\pi f_c \tau} \int_{-\infty}^{\infty} p\left(u + \frac{\tau}{2}\right) p^*\left(u - \frac{\tau}{2}\right) e^{-j2\pi\alpha u}\, du, \qquad \alpha = \frac{\nu}{T_0},\ \nu \in \mathbb{Z}. \tag{15}$$

The corresponding CSD is:

$$S_X^\alpha(f) = \frac{\sigma_a^2}{T_0} P\left(f - f_c + \frac{\alpha}{2}\right) P^*\left(f - f_c - \frac{\alpha}{2}\right), \qquad \alpha = \frac{\nu}{T_0}. \tag{16}$$

Here, $P(f)$ is the Fourier transform of $p(t)$. If $p(t)$ is an SRRC pulse with positive roll-off, the nontrivial cycle frequencies for this CSD occur at $\alpha = \pm R_s$, in addition to the ordinary $\alpha = 0$ component. Since this implementation searches only positive cycle frequencies, it evaluates candidates near $S_X^{R_s}(f)$. In the ideal model, the peak coordinates estimate (R_s, f_c) to within the estimator's cycle- and spectral-frequency resolutions. This behavior is illustrated in Figure 11.

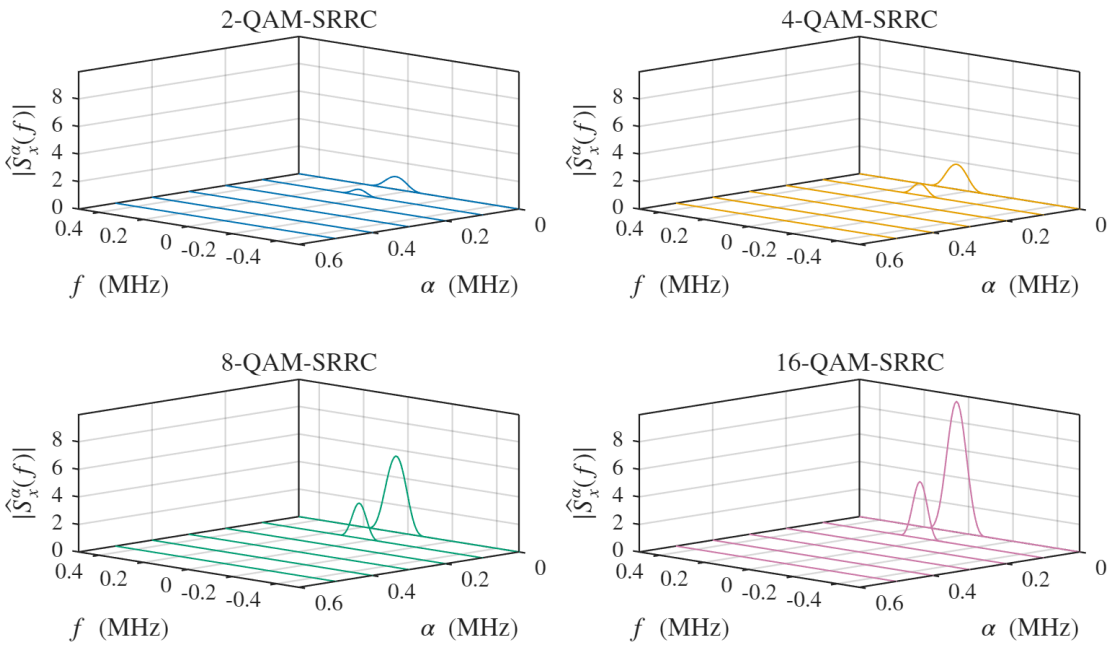

Figure 11: CSD magnitude estimates for independently generated 2-, 4-, 8-, and 16-QAM-SRRC waveforms with $R_s = 0.1$ MBd and $f_c = 0.05$ MHz. The minimum-distance-two constellations have symbol variances $\sigma_a^2 = 1$, 2, 6, and 10, respectively. The common vertical scale shows the shared second-order CSD shape and its amplitude proportional to σ_a^2. The $\alpha = 0$ slice is the ordinary PSD, and each nontrivial $\alpha = R_s$ slice has its maximum within one 7.8125-kHz DFT bin of f_c.

For the discrete CAF-DFT statistic used in this thesis, define the sequence in the kth channel as

$$z_k[n] = \left[\left(x[\cdot]e^{-j2\pi f_k(\cdot)/F_s}\right) * h[\cdot]\right][n]. \tag{17}$$

Its estimated zero-lag CAF at the vth cycle-frequency candidate is

$$\widehat{C}_x(\alpha_v, f_k) \triangleq \widehat{R}_{z_k}^{\alpha_v}[0] = \frac{1}{N}\sum_{n=0}^{N-1} |z_k[n]|^2 e^{-j2\pi\alpha_v n/F_s}. \tag{18}$$

Here, α_v is the vth physical cycle-frequency candidate in hertz, f_k is the kth physical CF candidate, and n is the zero-based sample index. The quantity $\widehat{C}_x(\alpha_v, f_k)$ is a channelized zero-lag CAF statistic—a filter-localized proxy for cyclic spectral

content—rather than the point CSD value $\widehat{S}_X^{\alpha_v}(f_k)$. On an N-point DFT-bin grid with one-based v, $\alpha_v = (v-1)F_s/N$, so the final kernel reduces to $e^{-j2\pi(v-1)n/N}$. More generally, an NCO can evaluate candidates that do not lie on that grid, so v is an array index rather than a frequency. A buffered, parallel realization of Equation (18) is shown in Figure 12; it adapts the mixer/filter search topology of the carrier-frequency sweep described in [5] while using a fixed $h[n]$ and no per-shift normalization.

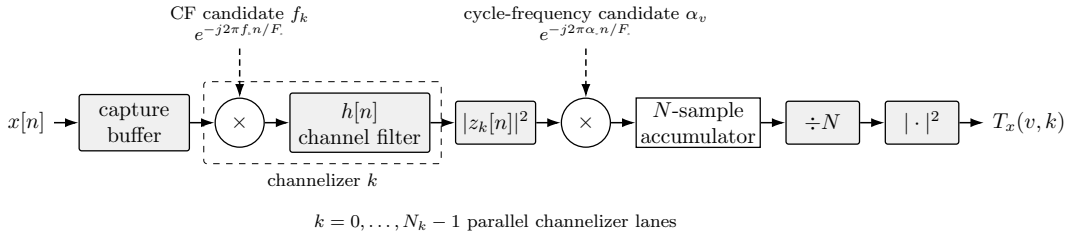

Figure 12: This buffered CAF-DFT reference realization frequency-shifts $x[n]$ by each physical CF candidate f_k and passes each shifted version through a low-pass filter, a process referred to as channelization. The magnitude squared produces the zero-lag autocorrelation sequence. Correlation with the cycle-frequency kernel at the physical candidate α_v, accumulation of N samples, and division by N produce $\widehat{C}_x(\alpha_v, f_k)$. Its magnitude squared is the nonnegative test statistic $T_x(v,k)$.

As shown in Figure 12, the CAF-DFT narrows the search to candidate SR-CF pairs for M-QAM-SRRC waveforms. Fixing $\tau = 0$ removes the lag dimension and therefore avoids the Fourier transform across lag required to form a full CSD. It does not eliminate the cycle-frequency correlation in Equation (18), which may be implemented by a DFT or by selected NCO kernels. This structure lends itself well to streaming processing.

2.4 Spectrum Sensing

Spectrum sensing is the process of periodically monitoring a specific frequency band to infer the presence or absence of users. Monitoring is typically performed by a secondary user (SU) to reuse inactive frequencies allocated or licensed to a primary

user (PU) [4, 6]. These allocated, inactive areas of a band are referred to as *spectral holes* [4]. In this section, we discuss several detection techniques and the strengths of CSP for spectrum sensing.

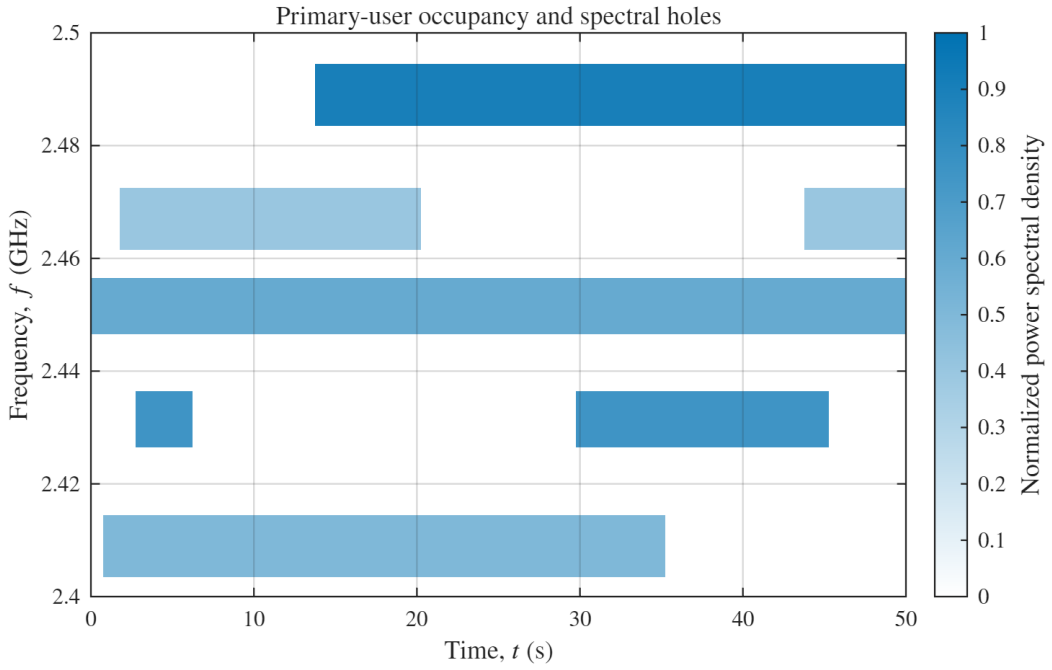

Figure 13: Conceptual time–frequency occupancy of five PU bands. White regions are spectral holes and blue regions indicate normalized PU power. An SU that maintains awareness of active PUs can hop among these holes and adapt its symbol rate to the currently available bandwidth.

2.4.1 The Binary Hypothesis Test

Let us consider an SU that is monitoring a narrow RF band for the presence of a PU. There are two hypotheses: under H_0, the PU is inactive; under H_1, the PU is active [6]. The received signal is written as follows:

$$H_0 : Y(t) = \eta(t) \tag{19}$$

$$H_1 : Y(t) = G(t) * X(t) + \eta(t), \tag{20}$$

where $X(t)$ denotes the transmitted signal from the primary user, $G(t)$ is the channel impulse response, $*$ denotes convolution, and $\eta(t)$ is zero-mean additive white Gaussian noise (AWGN) independent of $X(t)$. In the sampled model used later, $\eta[n]$ is represented by IID Gaussian samples. Exactly one of H_0 and H_1 is true, and the receiver selects a decision $\widehat{H}$ from the observed samples [6]. The four conditional decision probabilities are

1. Probability of correct rejection, $P_{\mathrm{CR}} = \Pr(\widehat{H} = H_0 \mid H_0)$: H_0 is true; choose H_0.
2. Probability of miss, $P_{\mathrm{M}} = \Pr(\widehat{H} = H_0 \mid H_1)$: H_1 is true; choose H_0.
3. Probability of false alarm, $P_{\mathrm{FA}} = \Pr(\widehat{H} = H_1 \mid H_0)$: H_0 is true; choose H_1.
4. Probability of detection, $P_{\mathrm{D}} = \Pr(\widehat{H} = H_1 \mid H_1)$: H_1 is true; choose H_1.

Outcomes P_{D} and P_{CR} correspond to correct decisions, whereas P_{M} and P_{FA} correspond to errors. If the SU transmits only after deciding H_0, these probabilities have the following physical interpretations:

1. P_{CR} is the probability that the SU correctly identifies an idle band and may transmit.
2. P_{M} is the probability that the SU declares an occupied band idle and may interfere with the PU.
3. P_{FA} is the probability that the SU declares an idle band occupied and forgoes an opportunity to transmit.
4. P_{D} is the probability that the SU correctly identifies an active PU and refrains from transmitting.

For simple hypotheses with known conditional densities, the Neyman–Pearson lemma states that a likelihood-ratio test maximizes P_{D} among tests whose P_{FA} does not exceed a specified level [6, 8]. Let $\mathbf{y}$ denote the observed sample vector. Its likelihood ratio is

$$\Lambda(\mathbf{y}) = \frac{p(\mathbf{y} \mid H_1)}{p(\mathbf{y} \mid H_0)}. \tag{21}$$

Comparing $\Lambda(\mathbf{y})$ with a threshold γ gives the likelihood-ratio test

$$\Lambda(\mathbf{y}) \underset{H_0}{\overset{H_1}{\gtrless}} \gamma. \tag{22}$$

If $\Lambda(\mathbf{y})$ exceeds γ, choose H_1; otherwise, choose H_0 [6]. Figure 14 illustrates this concept:

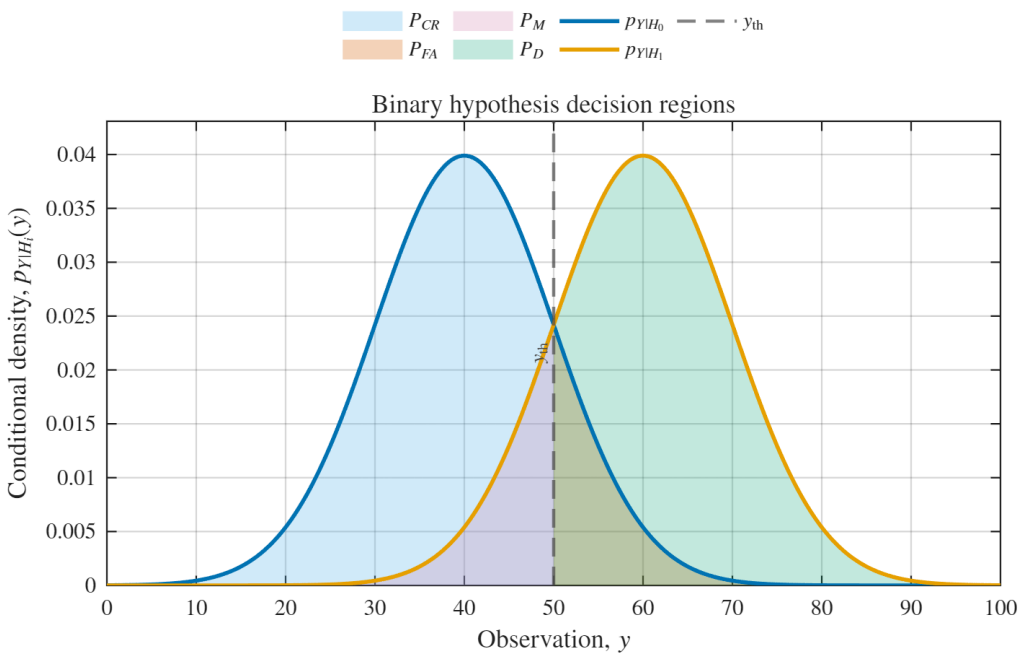

Figure 14: Conditional Gaussian observation densities with equal priors, equal decision costs, common standard deviation 10, and means 40 and 60. The observation threshold $y_{\mathrm{th}} = 50$ corresponds to likelihood-ratio threshold $\gamma = 1$ and gives $P_{\mathrm{FA}} = 0.1587$ and $P_{\mathrm{D}} = 0.8413$. Shading identifies the four decision regions.

2.4.2 Classical Detection Methods

There are many spectrum-sensing detection methods available. They are broadly categorized as *cooperative*—where coordination among many receivers is leveraged to detect a waveform—or *non-cooperative*—where the receiver detects based only on what it can sense [4]. In this subsection, we discuss three non-cooperative detection methods.

2.4.2.1 Matched Filter Detector

For a deterministic signal known to the receiver and observed in AWGN, the matched-filter test is optimal [4, 6]. The matched filter maximizes the output signal-to-noise ratio (SNR) by correlating the received signal with the known waveform and is often used within a coherent detector. Its performance degrades when the received waveform differs from the template or contains unmodeled interference [4].

2.4.2.2 Energy Detector

For a single-antenna sampled model in which the signal and noise samples are IID zero-mean Gaussian random variables and the noise power is known, energy detection is optimal [6]. The energy detector compares the accumulated received energy with a threshold based on the noise power, declaring a PU present when the threshold is exceeded [4, 6]. It is attractive when little prior information about the signal is available, but it does not classify signal features and is vulnerable to unknown or changing noise levels.

2.4.2.3 Cyclostationary Feature Detector

A cyclostationary feature detector (CFD) can distinguish a modulated signal from stationary noise by testing for nonzero-cycle features, often at lower SNR than an energy detector can support [4]. Let $Q_Y(\alpha_v, f_k)$ denote either a full-CSD value $S_Y^{\alpha_v}(f_k)$ or the channelized zero-lag CAF quantity $C_Y(\alpha_v, f_k)$ underlying Equation (18). For either feature, stationary AWGN gives the idealized population relations

$$H_0 : Q_Y(\alpha_v, f_k) = 0, \quad \alpha_v \neq 0, \tag{23}$$

$$H_1 : Q_Y(\alpha_v, f_k) \neq 0 \quad \text{for some } (\alpha_v, f_k). \tag{24}$$

These equalities describe the ideal nonzero-cycle feature. A finite-sample estimate $\widehat{Q}_y(\alpha_v, f_k)$ is generally not exactly zero under H_0 because of estimation variance and spectral leakage, so the decision requires a threshold. If the statistic exceeds that threshold at any f_k for a fixed α_v, the receiver declares cyclostationary behavior at that candidate rate. Under the assumed M-QAM-SRRC, single-feature model, a uniquely resolved maximizing f_k estimates the PU's center frequency.

2.4.3 Test Statistic and Detection Threshold

In a spectrum-sensing environment, the receiver may lack prior information about the signal and its activity probability [6]. For the illustrative detector used here, define the nonnegative pointwise test statistic $T_y(v, k)$ for each candidate (α_v, f_k) pair:

$$T_y(v, k) = \left|\widehat{Q}_y(\alpha_v, f_k)\right|^2. \tag{25}$$

The statistic still requires a threshold γ. Noise power is difficult to estimate in a realistic spectrum-sensing environment [6], so this demonstration instead normalizes the threshold to the estimated average received power,

$$\widehat{P}_y = \frac{1}{N} \sum_{n=0}^{N-1} |y[n]|^2. \tag{26}$$

The empirical threshold is $\gamma = \beta \widehat{P}_y^2$, where the dimensionless scale β controls sensitivity. It is not itself a calibrated false-alarm probability; such calibration would require the statistic's null distribution or Monte Carlo trials. The resulting test is

$$\left|\widehat{Q}_y(\alpha_v, f_k)\right|^2 \underset{H_0}{\overset{H_1}{\gtrless}} \beta\left(\frac{1}{N}\sum_{n=0}^{N-1}|y[n]|^2\right)^2. \tag{27}$$

Any nontrivial cycle-frequency slice containing a value above the threshold in Equation (27) is considered detected. Under the assumed signal model and a uniquely resolved peak, the largest value within each detected slice estimates its center frequency. Figure 15 illustrates this rule using a full-CSD estimate, $\widehat{Q}_y = \widehat{S}_Y$; Chapter III applies it to the channelized statistic $\widehat{Q}_y = \widehat{C}_y$.

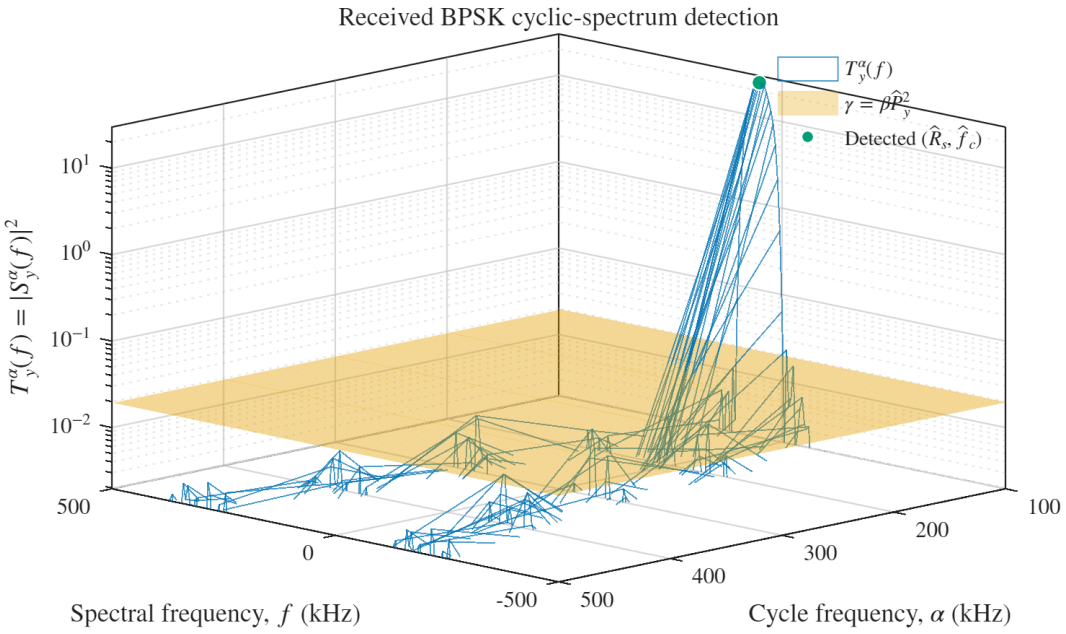

Figure 15: Full-CSD test statistic for a received BPSK-SRRC waveform with $R_s = 100$ kBd, $f_c = 50$ kHz, and requested $E_b/N_0 = 14$ dB. The nontrivial cycle-frequency slices are compared with the flat empirical threshold $\gamma = 10^{-2}\widehat{P}_y^2$. Only the 100-kHz slice exceeds the threshold, and its maximum occurs at 50 kHz.

CHAPTER III

IMPLEMENTATION

This work studies spectrum sensing for a prospective CubeSat cognitive radio (CR). For the M-QAM-SRRC signal model, the receiver searches a specified grid of candidate symbol rates (SRs) and center frequencies (CFs) and uses cyclostationary feature detection (CFD) to distinguish modeled signals from noise. A spectral correlation analyzer (SCA) realizes this detector. Conventional FFT-based SCAs obtain high throughput by evaluating many spectral channels in parallel. Their implementations buffer blocks of complex samples and intermediate transform results, so storage grows with both the capture length and the number of selected channels. This memory can compete with limited on-chip FPGA block RAM (BRAM) in a CubeSat-class implementation.

BRAM capacity is therefore a central design constraint. Reducing buffered storage improves fit within on-chip memory, while sample-by-sample time-domain processing trades the parallel throughput of an FFT-based analyzer for a deeply pipelined data path. The application focuses on detecting the SR and CF of M-QAM-SRRC waveforms, allowing the candidate search to be optimized for those cyclic features. For this blind SR–CF estimator, we borrow the communications term *streaming*. Thus, this design will be referred to as the *streaming SCA*, an FPGA architecture for resource-conscious signal processing.

3.1 Motivation

Orbital systems may use a single spacecraft, a constellation, or a distributed architecture such as a fractionated, clustered, or trailing formation. Distributed architectures coordinate multiple spacecraft, and CubeSats are one relevant small-spacecraft platform [1].

Crosslinks and relays create operating scenarios in which inter-satellite interference may need to be monitored. Spectrum sensing is one candidate mechanism for detecting occupied channels and informing an interference-avoidance response.

As discussed in the previous chapter, SCAs estimate cyclic spectral structure that can be used for spectrum sensing [7]. Several efficient digital SCA realizations use FFTs, sample blocks, and intermediate arrays whose storage requirements can be substantial [6, 7].

For a concrete scaling example, an SSCA intermediate array with $N = 2^{16}$ time samples and $N_p = 32$ selected frequency channels contains NN_p complex words [7]. At 16 bits for each in-phase and quadrature component, or 32 bits per complex word, that array alone requires $2^{16} \times 32 \times 32 = 67{,}108{,}864$ bits, or 67.1 Mbit, before control and buffering overhead. The implementation targets a ZedBoard with an XC7Z020CLG484-1, which provides 4.9 Mbit of block RAM [9]. The streaming architecture therefore replaces the large intermediate array with bounded pipeline state that fits the selected platform's memory budget.

3.2 The Streaming SCA

Let us assume that we wish to sense modeled waveforms present in a channel. As discussed in Chapter II, the single-term CAF-DFT form can evaluate selected frequency candidates without a block FFT. To create a low-memory design suitable for evaluation on the selected FPGA, we alter the CAF-DFT such that it:

- Evaluates only one pipelined (v, k) pair at a time, called a point. This is equivalent to one branch in Figure 12. If our SCA is well designed, the branches

can be reconfigured so that multiple instances can be used in parallel when the platform has extra resources.

- Avoids a full-capture BRAM buffer. A block-based realization buffers a capture so that every (v, k) pair is evaluated on the same samples. This design accepts a continuous input sample stream from the system-level data interface and evaluates successive candidate pairs over successive time intervals, reducing buffered storage for signals that remain stable over the search.

- Targets FPGA implementation so that the estimator can use deterministic pipelines and configurable parallelism within the platform's resource budget. Device qualification and mitigation are applied during system integration.

- Evaluates nonzero cycle frequencies $\alpha \neq 0$, which represent the cyclostationary features used by the detector; the zero-cycle-frequency case corresponds to the ordinary PSD.

- Uses a fixed two-tap low-pass filter (LPF), corresponding to taps $[0.5, 0.5]$. Koch's evaluated carrier-frequency detector systematically shifts each segment, dynamically configures the LPF bandwidth to the target symbol rate, and evaluates the CAC at that rate [5]. The streaming design is a serial hardware adaptation of that sweep; to avoid a reconfigurable coefficient bank, it uses the same two coefficients for every candidate.

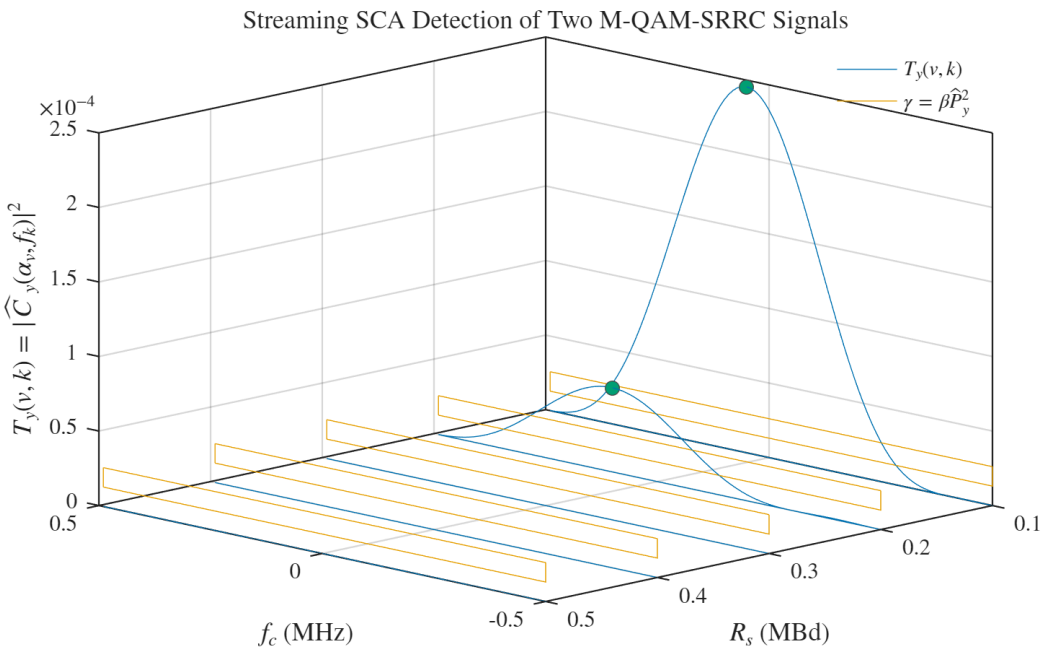

Figure 16: Streaming SCA test statistic $T_y(v,k) = |\widehat{C}_y(\alpha_v, f_k)|^2$ for two M-QAM-SRRC signals in a shared noise floor. The transmitted parameter pairs are $(R_s, f_c) = (100\text{ kBd}, 50\text{ kHz})$ and $(200\text{ kBd}, 100\text{ kHz})$. Both pairs lie on the estimator grid, both row maxima identify the transmitted CF, and both exceed the constant threshold $\gamma = \beta\widehat{P}_y^2$ with $\beta = 10^{-3}$. Green markers indicate the detected row maxima.

The streaming SCA was modeled in MATLAB to produce Figure 16. The two generated components are scaled so that a single receiver-noise realization gives the requested E_b/N_0 values of 14 dB and 9 dB. A common automatic-gain-control (AGC) scale factor then limits the composite waveform without changing either component's carrier-to-noise ratio. On the 10 kHz CF grid, the row maxima occur at the transmitted CFs of 50 kHz and 100 kHz; their test-statistic-to-threshold ratios are 9.41 and 2.16, respectively. These numerical values are reproducible because the model fixes its random-number seed. The executable MATLAB model is imported directly in Appendix A.

In exchange for this low memory use, the processing time is increased. With one (v,k) point evaluated at a time, no backpressure, and a sustained rate of one

valid sample per clock, an exhaustive search over N_v SR candidates and N_k CF candidates requires approximately NN_vN_k sample clocks, in addition to pipeline and control overhead. For fixed search spans, the processing delay therefore increases with capture length and with the number of candidates:

- SR step size: A smaller step produces more SR candidates and a longer search, whereas a larger step coarsens the grid and can increase parameter mismatch.
- CF step size: A smaller step produces more CF candidates and a longer search, whereas a larger step coarsens the grid and can increase parameter mismatch.
- Capture length N: A larger capture takes longer to process but provides a longer observation interval; a smaller capture reduces delay at the cost of a less stable finite-record estimate.

3.3 Digital Design

To describe the implementation of the streaming SCA as a digital circuit, we will walk through each component and its subcomponents. The signal-processing datapath uses custom VHDL modules and selected-frequency arithmetic in place of a vendor FFT core. MATLAB and Xilinx Vivado support modeling, simulation, synthesis, and implementation. The tables define the configured external interfaces used by the architecture; propagation-only width and latency parameters remain internal to their corresponding pipeline stages.

Interface and naming conventions. The interface tables use descriptive labels: `I_WIDTH` and `Q_WIDTH` for real and imaginary component widths, `A_WIDTH` and `B_WIDTH` for multiplier-operand widths, and `MSB_TRUNC_BITS` and `LSB_ROUND_BITS` for the bit counts removed at each end of a fixed-point word. These labels correspond to the VHDL generics `LengthA`, `LengthB`, `BitstoTrunc`, and `BitstoRound`. References to source identifiers retain their exact spelling; displayed frequency-control-word names consistently use `FCW`.

The stream signal roles mirror AXI4-Stream: data, valid, and the final-sample tag travel downstream, while ready travels upstream. The datapath specializes those roles to a continuous-rate, fixed-latency interface. During each N-sample record, `SampleInValid` is asserted on every clock and downstream `SampleOutReady` is held high. In sample-preserving stages, data and their valid and final-sample tags advance together at a fixed latency. At an integrate-and-dump boundary, every accepted input contributes to the record state and one result, valid tag, and final-sample tag are emitted after the terminal input reaches that boundary. The module-local `SampleIn` and `SampleOut` names identify this continuous-rate core interface; an elastic AXI4-Stream wrapper can map them to `s_axis` and `m_axis` channels while supplying stall-hold behavior at the protocol boundary.

The configured CFE datapath uses `TAP_COUNT=2`, giving the moving-average response $h = [0.5, 0.5]$. Its selected-frequency symbol-rate stage follows the streaming architecture described below, while the arithmetic and control interfaces are implemented as custom VHDL pipelines. The discussion occasionally refers to VHDL functions, such as `resize()`; however, the signal-processing description is otherwise language-agnostic.

3.3.1 Center Frequency Estimator

The purpose of the center frequency estimator (CFE) is to produce a test-statistic surface over the SR–CF candidates. Let v and k index user-defined candidate arrays with physical frequencies α_v and f_k, respectively. For an input sequence $x[n]$, define

$$z_k[n] = \left[\left(x[\cdot]e^{-j2\pi f_k(\cdot)/F_s}\right) * h[\cdot]\right][n], \qquad \widehat{C}_x(\alpha_v, f_k) = \frac{1}{N}\sum_{n=0}^{N-1} |z_k[n]|^2 e^{-j2\pi\alpha_v n/F_s}.$$

The center-frequency control word configures a downconverting local oscillator for the factor $e^{-j2\pi f_k n/F_s}$. Next, an LPF module smooths the mixed signal with the fixed

taps $[0.5, 0.5]$. The SRE then evaluates the zero-lag CAF coefficient $\widehat{C}_x(\alpha_v, f_k) = \widehat{R}_{z_k}^{\alpha_v}[0]$, and the final magnitude-squared operation produces one point of the test statistic, $T_x(v,k) = |\widehat{C}_x(\alpha_v, f_k)|^2$. Thus, v and k are candidate-array indices rather than frequencies themselves. As explained in Chapter II, $\widehat{C}_x$ is interpreted as the filter-localized cyclic statistic associated with f_k.

For each candidate pair (α_v, f_k), the controller holds both frequency-control words constant for one N-sample record. Each sample is downconverted by f_k, filtered by $h[n]$, converted to a nonnegative magnitude-squared value, mixed by α_v, and accumulated. The final sample is included exactly once in the record result. After normalization by N, the magnitude squared of the complex sum forms $T_x(v,k)$. After the preceding record boundary has traversed the fixed-latency pipeline, the controller supplies an inter-record invalid bubble that inserts zero into the FIR's one-sample delay. It then applies the next candidate pair and accepts its first sample with zero FIR state.

The thesis configuration specializes the NCO to a 16-bit phase accumulator with two quadrant bits and a 14-bit quarter-wave lookup address. An integer control word q represents $qF_s/2^{16}$ modulo F_s, giving a 15.2588-Hz quantum at $F_s = 1$ MHz. Signed center-frequency codes represent $[-F_s/2, F_s/2 - F_s/2^{16}]$. Unsigned symbol-rate codes 0 through 32767 retain their positive interpretation at the signed NCO boundary, while code 32768 represents the aliased Nyquist endpoint. In continuous-rate operation, the oscillator advances one control-word step on every sample clock, so oscillator phase index and sample index advance together. If the cycle-frequency candidates are restricted to an N-point DFT grid, MATLAB's one-based index satisfies $\alpha_v = (v-1)F_s/N$; the NCO can also evaluate candidates off that grid subject to its 15.2588-Hz quantization.

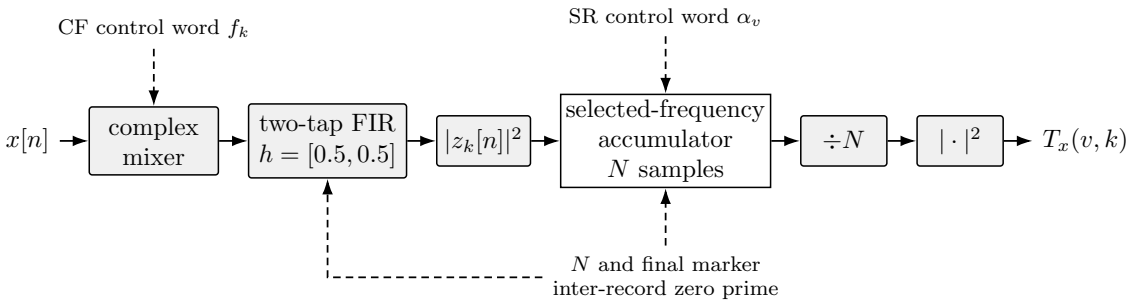

Figure 17: Functional architecture of the CFE module. Solid arrows carry sampled data; dashed arrows carry configuration and block control.

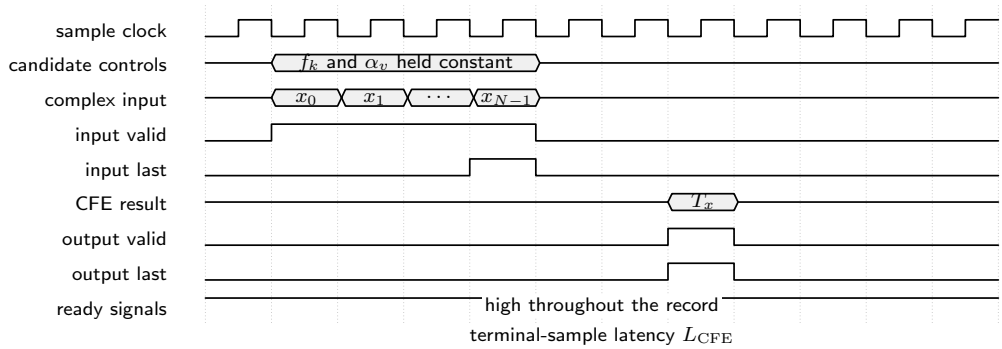

Figure 18: Symbolic continuous-rate timing of the CFE module; both control words remain fixed for each N-sample record, and one statistic is emitted after the terminal sample.

Table III: Configured CFE interface.

Interface Item	**Direction**	**Data Type**	**Description**
N_SAMPLES	Generic	natural	Configured record length; `SampleInLast` marks sample $N-1$.
BIT_WIDTH	Generic	natural	Sample width: one sign bit and BIT_WIDTH-1 fractional bits.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
CenterFrequency FCW	Input	signed(15 downto 0)	Signed 16-bit CF control word.
SymbolRateFCW	Input	unsigned(15 downto 0)	Unsigned 16-bit SR control word.
SampleIn_I	Input	signed(BIT_WIDTH-1 downto 0)	Real component of sample.
SampleIn_Q	Input	signed(BIT_WIDTH-1 downto 0)	Imaginary component of sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
OverflowStatus	Output	std_logic_vector(3 downto 0)	[3]: CF mixer I [2]: CF mixer Q [1]: SR mixer I [0]: SR mixer Q
SampleOut	Output	unsigned(2*BIT_WIDTH-1 downto 0)	Nonnegative magnitude-squared result.

3.3.2 Symbol Rate Estimator

The purpose of the SR estimator (SRE) is to estimate the correlation between a waveform and the candidate SR cycle frequency α_v. Its computation is the finite-

record CAF estimate evaluated at $\tau = 0$:

$$\widehat{R}_X^{\alpha_v}[0] = \frac{1}{N} \sum_{n=0}^{N-1} |X[n]|^2 e^{-j2\pi\alpha_v n/F_s}.$$

The instantaneous-power module computes the nonnegative value $|X[n]|^2$, and the selected-frequency Fourier-sum module evaluates the displayed zero-lag CAF expression at α_v. For Q1.15 input components, $P = I^2 + Q^2$ lies in $[0, 2]$ and has a full-precision unsigned 32-bit representation with 30 fractional bits. The format boundary into the signed Fourier mixer therefore preserves the magnitude bit by either zero-extending into a wider signed word or by explicitly rescaling and saturating. One conforming 16-bit mapping is a saturated Q1.15 representation of $P/2$. This scales the resulting complex CAF coefficient by one half and the downstream magnitude-squared statistic and matching threshold by one quarter; those factors are carried consistently. Every power sample remains nonnegative throughout the interface.

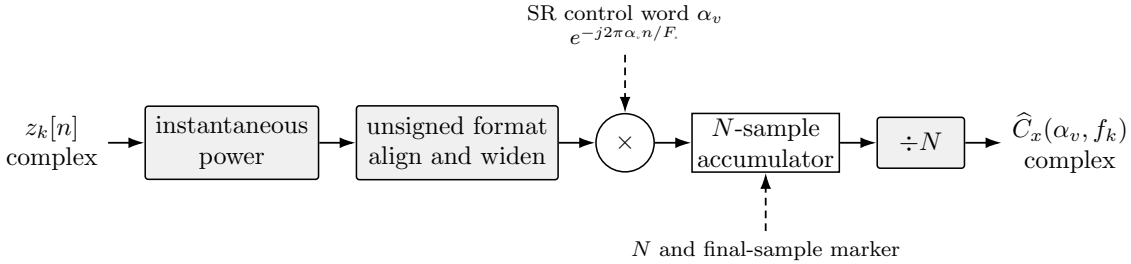

Figure 19: Functional architecture of the SRE module, including the nonnegative fixed-point format boundary after instantaneous power.

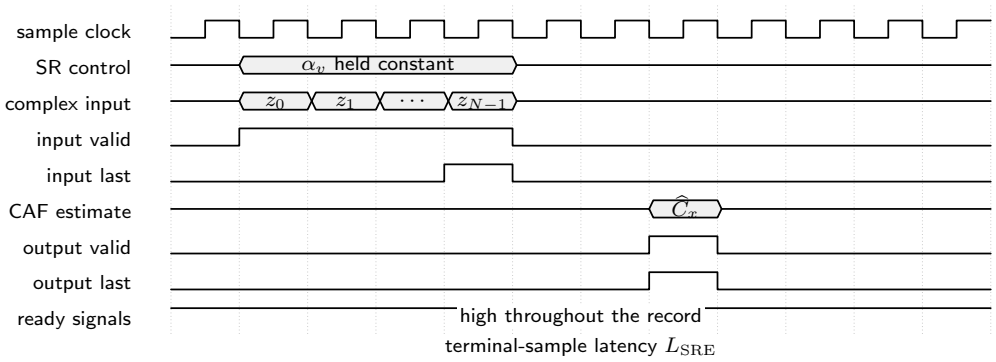

Figure 20: Symbolic continuous-rate timing of the SRE module with an aligned terminal result, valid tag, and final-sample tag.

Table IV: Configured SRE interface.

Interface Item	**Direction**	**Data Type**	**Description**
N_SAMPLES	Generic	natural	Configured record length; `SampleInLast` marks sample $N-1$.
BIT_WIDTH	Generic	natural	Sample width: one sign bit and BIT_WIDTH-1 fractional bits.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
OscillatorFCW	Input	unsigned(15 downto 0)	Unsigned 16-bit SR control word.
SampleIn_I	Input	signed(BIT_WIDTH-1 downto 0)	Real component of sample.
SampleIn_Q	Input	signed(BIT_WIDTH-1 downto 0)	Imaginary component of sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
OverflowStatus	Output	std_logic_vector(1 downto 0)	[1]: mixer I [0]: mixer Q
SampleOut_I	Output	signed(BIT_WIDTH-1 downto 0)	Real component of result.
SampleOut_Q	Output	signed(BIT_WIDTH-1 downto 0)	Imaginary component of result.

3.3.3 Discrete Fourier Transform

The DFT is used to perform frequency-domain analysis of discrete-time signals. This implementation evaluates a finite-record Fourier sum at one selected physical frequency f_0 at a time:

$$\widehat{X}(f_0) = \frac{1}{N} \sum_{n=0}^{N-1} X[n] e^{-j2\pi f_0 n/F_s}.$$

The corresponding normalized N-point DFT coefficient $\widehat{X}[\ell]$ is the special case $f_0 = \ell F_s/N$ for the zero-based bin $0 \leq \ell < N$. The oscillator frequency-control word selects f_0, the mixer forms the complex products $X[n]e^{-j2\pi f_0 n/F_s}$, and separate real and imaginary accumulators sum exactly N products. The final marked sample is included before the completed sums are normalized, quantized, and emitted. The quarter-wave lookup-table reconstruction can introduce a constant NCO initial-phase factor, but that factor cancels in the final magnitude squared. For the SRE, $f_0 = \alpha_v$; when the candidates lie on the DFT grid, $\ell = v - 1$ under MATLAB's one-based indexing.

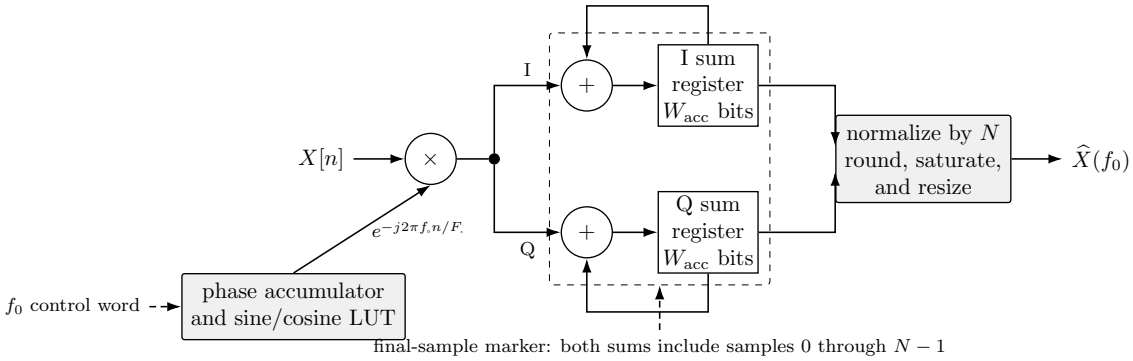

Figure 21: Selected-frequency Fourier-sum architecture. Both accumulators include the final accepted sample before normalization.

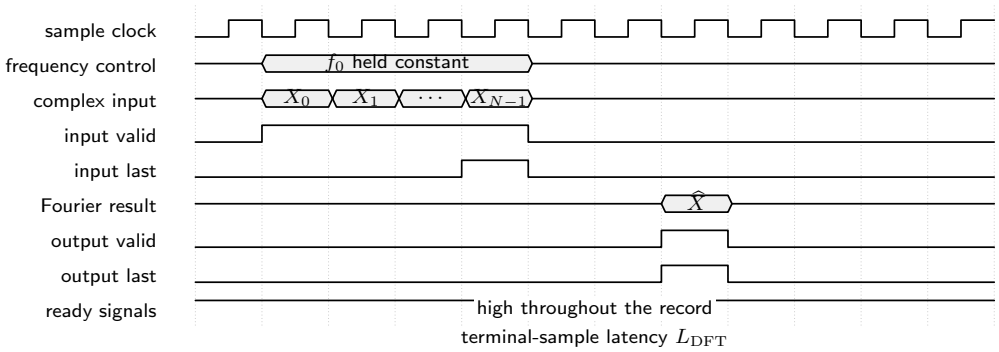

Figure 22: Symbolic continuous-rate timing of the selected-frequency DFT module; the normalized Fourier result follows the terminal input sample by fixed latency.

Table V: Configured DFT interface.

Interface Item	**Direction**	**Data Type**	**Description**
N_SAMPLES	Generic	natural	Configured record length; `SampleInLast` marks sample $N-1$.
BIT_WIDTH	Generic	natural	Sample width: one sign bit and BIT_WIDTH-1 fractional bits.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
OscillatorFCW	Input	signed(15 downto 0)	Signed 16-bit frequency control word.
SampleIn_I	Input	signed(BIT_WIDTH-1 downto 0)	Real component of sample.
SampleIn_Q	Input	signed(BIT_WIDTH-1 downto 0)	Imaginary component of sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
OverflowStatus	Output	std_logic_vector(1 downto 0)	[1]: mixer I [0]: mixer Q
SampleOut_I	Output	signed(BIT_WIDTH-1 downto 0)	Real component of result.
SampleOut_Q	Output	signed(BIT_WIDTH-1 downto 0)	Imaginary component of result.

3.3.4 Frequency Mixer

Frequency translation is performed by multiplying the input by a complex local oscillator generated by a numerically controlled oscillator (NCO). For a positive-frequency component at f_1, multiplication by the downconverting oscillator at f_{LO} gives

$$e^{j2\pi f_1 n/F_s}\, e^{-j2\pi f_{\mathrm{LO}} n/F_s} = e^{j2\pi (f_1 - f_{\mathrm{LO}}) n/F_s}.$$

When $f_{\mathrm{LO}} = f_1$, the component is translated to zero frequency. The implementation generates the oscillator with a phase accumulator and a sine/cosine lookup table; the frequency-control word sets its phase increment. In continuous-rate operation, the phase accumulator advances once per sample clock and the phase index therefore remains aligned with the consecutive sample index n. An elastic interface gates the phase update with the sample-accept event so that a stalled transaction preserves both the sample and its corresponding oscillator phase.

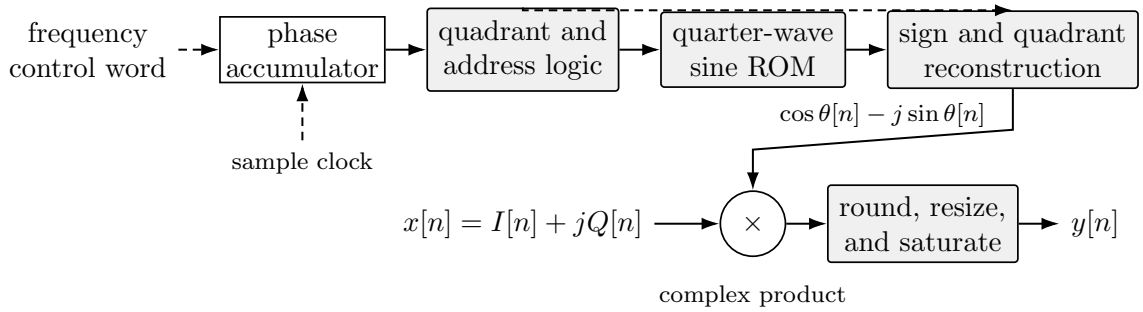

Figure 23: Functional architecture of the frequency mixer. The NCO phase advances once per sample clock during continuous-rate operation.

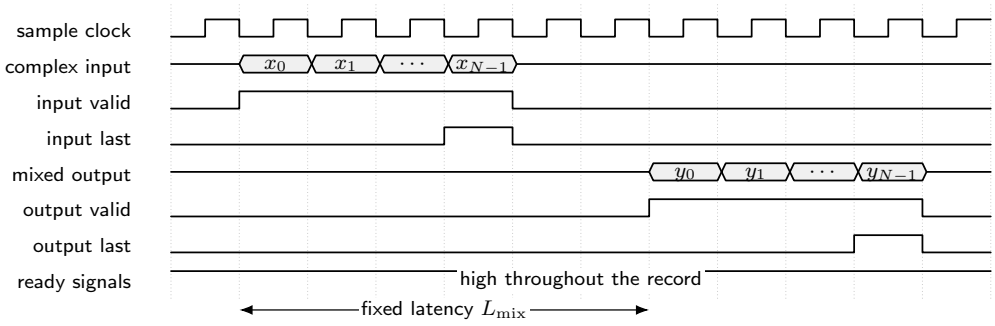

Figure 24: Symbolic continuous-rate timing of the frequency mixer module. Data, valid, and final-sample tags share the same fixed latency.

Table VI: Configured frequency-mixer interface.

Interface Item	**Direction**	**Data Type**	**Description**
BIT_WIDTH	Generic	natural	Sample width: one sign bit and BIT_WIDTH-1 fractional bits.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
OscillatorFCW	Input	signed(15 downto 0)	Signed 16-bit frequency control word.
SampleIn_I	Input	signed(BIT_WIDTH-1 downto 0)	Real component of sample.
SampleIn_Q	Input	signed(BIT_WIDTH-1 downto 0)	Imaginary component of sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
OverflowStatus	Output	std_logic_vector(1 downto 0)	[1]: I [0]: Q
SampleOut_I	Output	signed(BIT_WIDTH-1 downto 0)	Real component of mixed sample.
SampleOut_Q	Output	signed(BIT_WIDTH-1 downto 0)	Imaginary component of mixed sample.

3.3.5 Low-Pass Filter

A finite impulse response (FIR) filter has a finite-duration impulse response. Filter coefficients are calculated from the desired properties of the filter in MATLAB or Python. An L-tap FIR implements the convolution sum

$$Y[n] = \sum_{i=0}^{L-1} b[i]X[n-i].$$

Here, $b[i]$ are the L coefficients, and $X[n]$ is the input sequence. A causal FIR with L taps has order $L-1$. This design uses the two-tap, first-order moving-average low-pass filter $b[0] = b[1] = 0.5$. Each candidate record uses the zero-state convention $X[-1] = 0$. Within a record, the delay stores the preceding sample. Between records, at least one clock with `SampleInValid` deasserted inserts zero into the one-sample delay and inserts a deasserted valid tag for that corresponding delayed transaction; an earlier pipeline transaction may still emerge on the same clock. The next record therefore begins from zero state.

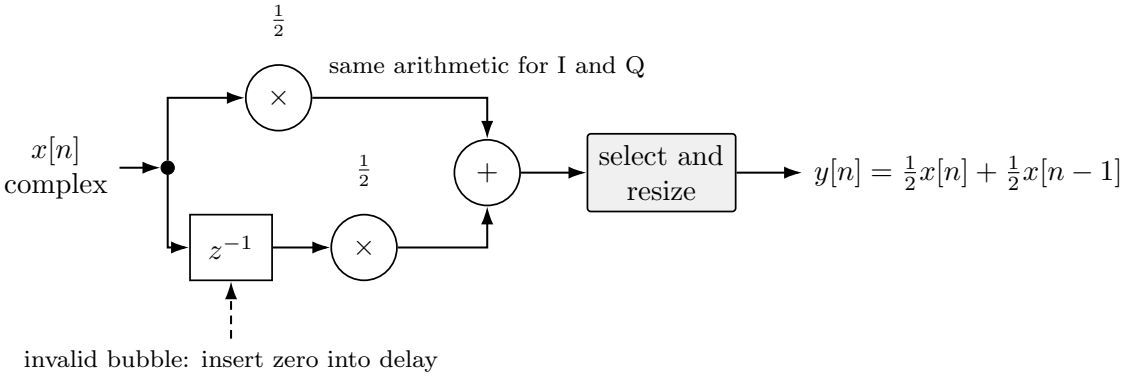

Figure 25: Configured two-tap complex low-pass filter with coefficients $[0.5, 0.5]$.

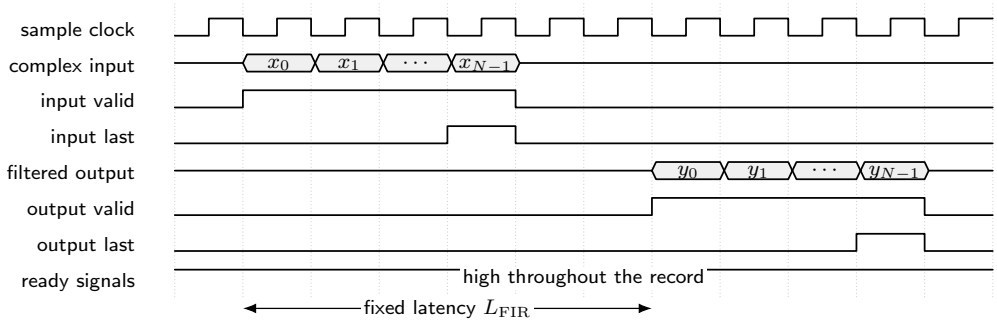

Figure 26: Symbolic continuous-rate timing of the two-tap low-pass filter module. Data, valid, and final-sample tags remain aligned.

Table VII: Configured low-pass-filter interface.

Interface Item	**Direction**	**Data Type**	**Description**
TAP_COUNT	Config.	natural	Number of coefficients.
TAP_WIDTH	Config.	natural	Coefficient width: one sign bit and TAP_WIDTH-1 fractional bits.
BIT_WIDTH	Config.	natural	Sample width: one sign bit and BIT_WIDTH-1 fractional bits.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
Coeffs	Input	CoeffArray(TAP_COUNT-1 downto 0)	TAP_COUNT signed coefficients, with $b[0] = b[1] = 0.5$ in this application.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
SampleIn_I	Input	signed(BIT_WIDTH-1 downto 0)	Real component of sample.
SampleIn_Q	Input	signed(BIT_WIDTH-1 downto 0)	Imaginary component of sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
SampleOut_I	Output	signed(BIT_WIDTH-1 downto 0)	Real component of filtered sample.
SampleOut_Q	Output	signed(BIT_WIDTH-1 downto 0)	Imaginary component of filtered sample.

3.3.6 Instantaneous Power

This module computes the pointwise magnitude squared of a complex sample. It supplies the $\tau = 0$ product $X[n]X^*[n]$ used by the zero-lag AF and CAF:

$$\begin{aligned} P_X[n] &= X[n]X^*[n] \\ &= |X[n]|^2 \\ &= \Re\{X[n]\}^2 + \Im\{X[n]\}^2. \end{aligned}$$

The real and imaginary components are squared in parallel and then added. This pointwise identity is not Parseval's theorem, which instead relates a sum or integral of squared magnitudes in the time domain to one in the frequency domain.

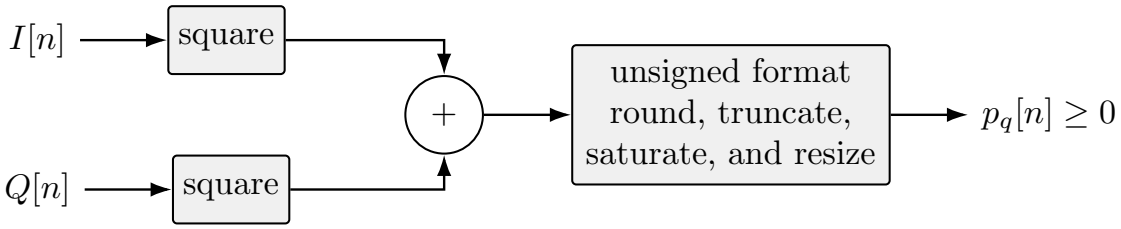

Figure 27: Instantaneous-power architecture with a nonnegative fixed-point output.

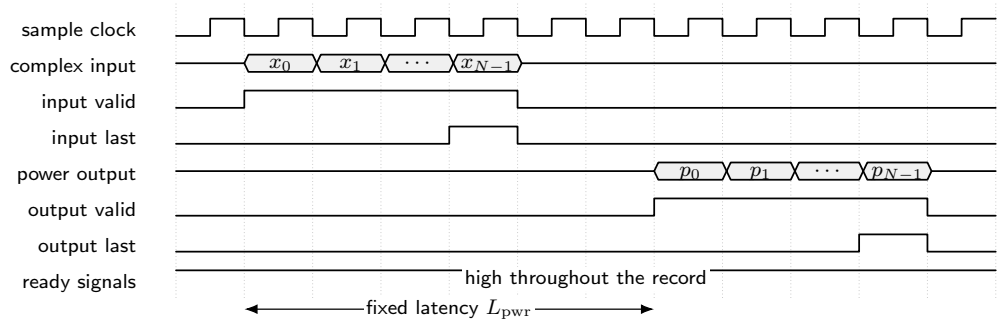

Figure 28: Symbolic continuous-rate timing of the instantaneous-power module. Each accepted complex input produces one nonnegative output after fixed latency.

Table VIII: Configured instantaneous-power interface.

Interface Item	**Direction**	**Data Type**	**Description**
I_WIDTH	Generic	natural	Bit width of the real input.
Q_WIDTH	Generic	natural	Bit width of the imaginary input.
LATENCY	Generic	natural	Pipeline latency in clock cycles.
MSB_TRUNC_BITS	Generic	natural	Number of most-significant bits truncated.
LSB_ROUND_BITS	Generic	natural	Number of least-significant bits removed after rounding.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
SampleIn_I	Input	signed(I_WIDTH-1 downto 0)	Real component of sample.
SampleIn_Q	Input	signed(Q_WIDTH-1 downto 0)	Imaginary component of sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
SampleOut	Output	unsigned(2*max(I_WIDTH, Q_WIDTH)-1- LSB_ROUND_BITS- MSB_TRUNC_BITS downto 0)	Nonnegative magnitude-squared output.

3.3.7 Complex Multiplier

Let the two complex operands be $X_A[n] = I_A[n] + jQ_A[n]$ and $X_B[n] = I_B[n] + jQ_B[n]$. Their product can be expressed in terms of the corresponding real and

imaginary components as follows:

$$\Re\{X_A[n]X_B[n]\} = I_A[n]I_B[n] - Q_A[n]Q_B[n],$$
$$\Im\{X_A[n]X_B[n]\} = I_A[n]Q_B[n] + Q_A[n]I_B[n].$$

Thus, the multiplication of $X_A[n]$ and $X_B[n]$ requires four real multiplications, one real addition, and one real subtraction.

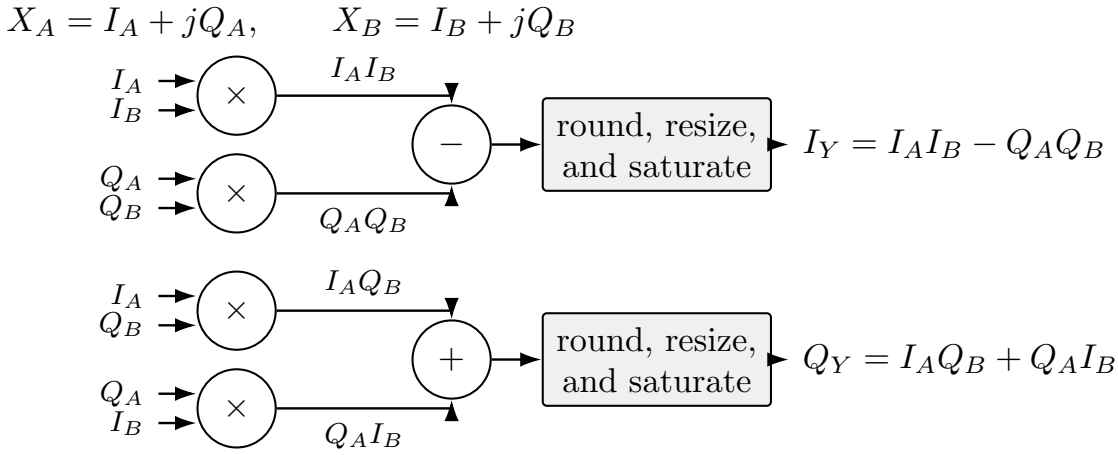

Figure 29: Complex-multiplier architecture with explicit real and imaginary arithmetic.

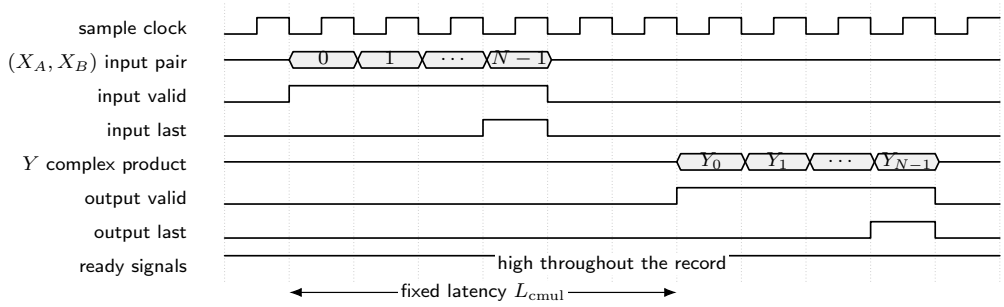

Figure 30: Symbolic continuous-rate timing of the complex-multiplier module. Each aligned input pair produces one complex product.

Table IX: Configured complex-multiplier interface.

Interface Item	**Direction**	**Data Type**	**Description**
A_WIDTH	Generic	natural	Operand-A component width.
B_WIDTH	Generic	natural	Operand-B component width.
LATENCY	Generic	natural	Pipeline latency in clock cycles.
MSB_TRUNC_BITS	Generic	natural	Most-significant bits truncated.
LSB_ROUND_BITS	Generic	natural	Least-significant bits removed after rounding.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Active-high synchronous reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	Input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
SampleIn_A_I	Input	signed(A_WIDTH-1 downto 0)	Operand-A real component.
SampleIn_A_Q	Input	signed(A_WIDTH-1 downto 0)	Operand-A imaginary component.
SampleIn_B_I	Input	signed(B_WIDTH-1 downto 0)	Operand-B real component.
SampleIn_B_Q	Input	signed(B_WIDTH-1 downto 0)	Operand-B imaginary component.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	Output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
SampleOut_I	Output	signed(A_WIDTH+B_WIDTH-LSB_ROUND_BITS-MSB_TRUNC_BITS downto 0)	Real component of product.
SampleOut_Q	Output	signed(A_WIDTH+B_WIDTH-LSB_ROUND_BITS-MSB_TRUNC_BITS downto 0)	Imaginary component of product.

3.3.8 Multiplier

There are many real multipliers used throughout this design. The module provides a continuous-rate, fixed-latency pipeline together with explicit bit-width, truncation, saturation, and rounding controls. Data, valid, and final-sample metadata advance through corresponding delay stages, while downstream ready is held high for the characterized operating mode. Mathematically, this module performs the multiplication of two real values, expressed as:

$$Y[n] = X_1[n]X_2[n]. \tag{28}$$

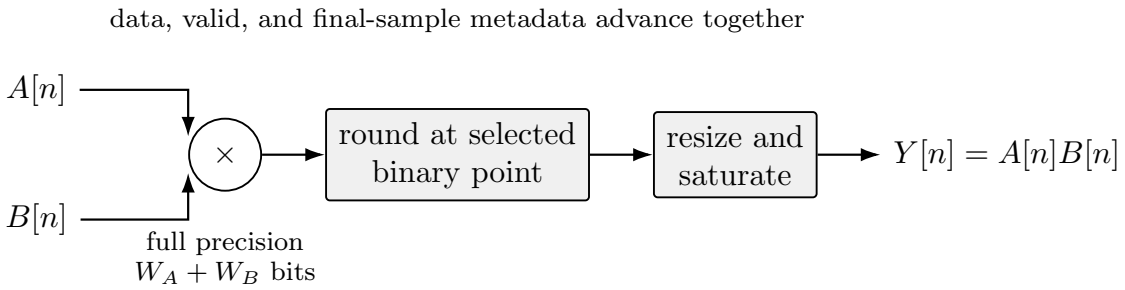

Figure 31: Fixed-point multiplier architecture with explicit rounding and saturation stages.

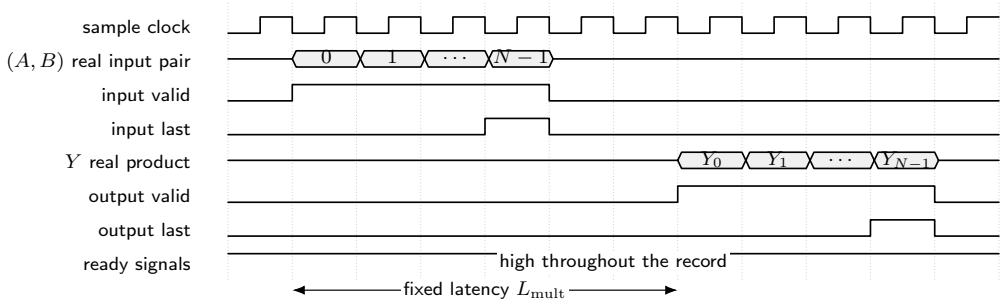

Figure 32: Symbolic continuous-rate timing of the fixed-point multiplier module. The product and its valid and final-sample tags share the configured latency.

Table X: Configured multiplier interface.

Interface Item	**Direction**	**Data Type**	**Description**
A_WIDTH	Generic	natural	Bit width of operand A.
B_WIDTH	Generic	natural	Bit width of operand B.
LATENCY	Generic	natural	Pipeline latency in clock cycles.
MSB_TRUNC_BITS	Generic	natural	Number of most-significant bits truncated.
LSB_ROUND_BITS	Generic	natural	Number of least-significant bits removed after rounding.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with the final input sample.
SampleIn_A	Input	signed(A_WIDTH-1 downto 0)	Real-valued operand A.
SampleIn_B	Input	signed(B_WIDTH-1 downto 0)	Real-valued operand B.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
SampleOut	Output	signed(A_WIDTH+B_WIDTH-1 -LSB_ROUND_BITS- MSB_TRUNC_BITS downto 0)	Real-valued product.

3.3.9 Accumulator

The accumulator implements the running sum used by each single-frequency Fourier evaluation and participates in the integrate-and-dump flow control between sample sets. For a sequence whose first accepted sample has local index $n = 0$, the recurrence is

$$S[n] = \begin{cases} X[n], & n = 0, \\ S[n-1] + X[n], & n > 0 \end{cases} . \tag{29}$$

Here, $X[n]$ is an input sample and $S[n]$ is the running sum within the current record. The module itself is real-valued; the DFT instantiates separate accumulators for the in-phase and quadrature components. On each valid sample, the adder first forms $S_{\text{next}} = S + X[n]$, with the first sample using a cleared stored sum. An ordinary sample stores S_{next}. A sample carrying `SampleInLast` stores no residual value: it emits the normalized S_{next}, asserts the aligned output-valid and final-sample tags, and clears the stored sum for the next record. This end-of-record convention includes each of the N samples exactly once.

For a signed W-bit input and a record of N samples, the full-range accumulator width is

$$W_{\text{acc}} = W + \lceil \log_2 N \rceil .$$

A 16-bit, 4096-sample configuration therefore uses a 28-bit sum and a 12-bit normalization shift. A 16-bit, 65,536-sample configuration uses a 32-bit sum and a 16-bit shift. For power-of-two N, a sign-aware rounding adjustment followed by selection of the corresponding upper bits implements rounded division by N; for other record lengths, normalization uses a constant reciprocal or divider with guard bits before the same rounding and saturation stage. The configured `N_SAMPLES` value establishes the counter limit, accumulator width, and normalization factor, while `SampleInLast`

is asserted with count $N-1$ as the protocol-level end-of-record tag. An invalid input clock leaves the sample-indexed state unchanged and inserts deasserted valid and final-sample tags into the fixed-latency tag path; a previously accepted sample or terminal result may still emerge on that clock according to the pipeline latency.

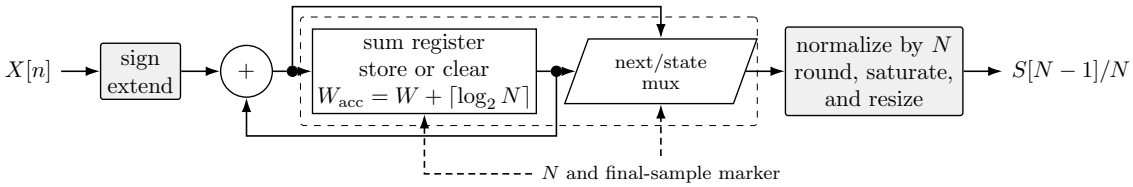

Figure 33: Block-synchronous accumulator architecture. The final accepted sample is included before the terminal sum is normalized and emitted.

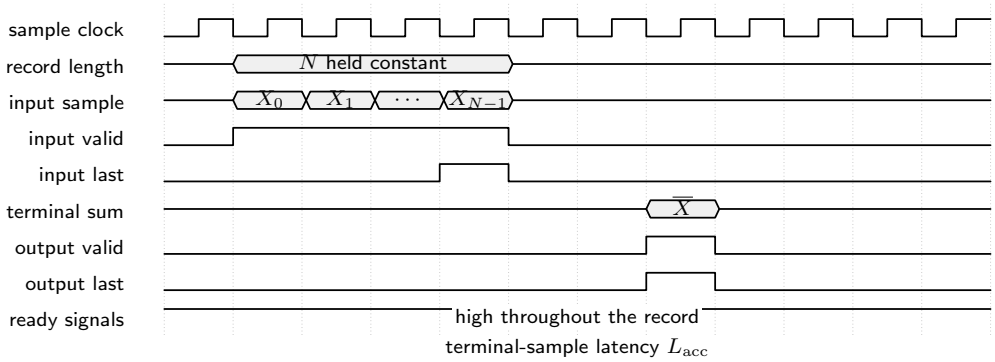

Figure 34: Symbolic block-synchronous timing of the accumulator module. The final-sample marker accompanies sample $N-1$, and one normalized terminal result is emitted after that sample has been included.

Table XI: Configured accumulator interface.

Interface Item	**Direction**	**Data Type**	**Description**
N_SAMPLES	Generic	natural	Record length used for counting, width, and normalization.
BIT_WIDTH	Generic	natural	Sample width: one sign bit and BIT_WIDTH-1 fractional bits.
Clock	Input	std_logic	Synchronous clock.
Reset	Input	std_logic	Synchronous, active-high reset.
SampleOutReady	Input	std_logic	Downstream status input; held high during continuous-rate operation.
SampleInValid	Input	std_logic	High when input is valid.
SampleInLast	Input	std_logic	End-of-record tag asserted with sample $N-1$.
SampleIn	Input	signed(BIT_WIDTH-1 downto 0)	Real-valued input sample.
SampleInReady	Output	std_logic	Latency-aligned upstream status for the continuous-rate interface.
SampleOutValid	Output	std_logic	High when output is valid.
SampleOutLast	Output	std_logic	End-of-record tag aligned with the associated output.
SampleOut	Output	signed(BIT_WIDTH-1 downto 0)	Normalized completed-record sum containing all N samples.

CHAPTER IV

PERFORMANCE CHARACTERIZATION

This chapter characterizes the streaming SCA's ability to sense and estimate the CF and SR of signals in a monitored spectrum. First, a Python model generates an M-QAM-SRRC waveform, applies the software channel, quantizes the in-phase and quadrature (IQ) samples, and saves them to a file. A VHDL testbench reads that file, evaluates the channelized zero-lag CAF statistic $T_y(v,k) = |\widehat{C}_y(\alpha_v, f_k)|^2$ over user-defined requested SR and CF candidate grids $\mathcal{R}$ and $\mathcal{F}$, and saves the results. MATLAB then plots the statistic and the constant threshold $\gamma = \beta \widehat{P}_y^2$. For each rate candidate $\alpha_v \in \mathcal{R}$, the selected CF is the $f_k \in \mathcal{F}$ having the largest statistic, and that row exceeds the threshold when its maximum is greater than γ. Table XIV lists the selected peak pair or pairs in nominal physical units. Each entry represents a row maximum, while v and k serve as candidate-array indices.

4.1 Experiments

Characterization begins with baseline transmitter and receiver parameters that yield a clear threshold crossing. Each subsequent experiment varies one listed parameter relative to that reference, covering modulation order, roll-off, input rate, center frequency, noise, sample rate, capture length, word length, and candidate-grid spacing. The parameters evaluated are shown in Table XII and Table XIII.

Table XII: Python transmitter and channel.
Here, $F_s = 1$ MHz except in Experiment VII, where $F_s = 10$ MHz; r is the roll-off factor. In the two-signal Experiments XII and XIII, the scalar E_b/N_0 applies to both components.

Expt.	$\boldsymbol{M}$	$\boldsymbol{r}$	$\boldsymbol{R_s}$ **(MBd)**	$\boldsymbol{f_c}$ **(MHz)**	$\frac{\boldsymbol{E_b}}{\boldsymbol{N_0}}$ **(dB)**
I	4	0.35	0.10	0.05	14.00
II	16	0.35	0.10	0.05	14.00
III	4	0.20	0.10	0.05	14.00
IV	4	0.35	0.01	0.05	14.00
V	4	0.35	0.10	−0.05	14.00
VI	4	0.35	0.10	0.05	9.00
VII	4	0.35	0.10	0.05	14.00
VIII	4	0.35	0.10	0.05	14.00
IX	4	0.35	0.10	0.05	14.00
X	4	0.35	0.10	0.05	14.00
XI	4	0.35	0.10	0.05	14.00
XII	4; 16	0.35; 0.35	0.10; 0.25	−0.20; 0.20	14.00
XIII	4; 16	0.35; 0.35	0.10; 0.25	0.05; 0.10	14.00

Table XIII: VHDL receiver.
Here, BW is the bit width, and $\mathcal{R}$ and $\mathcal{F}$ are inclusive requested (nominal) physical search grids. The notation $[a : s : b]$ means values from a through b in steps of s.

Expt.	$\boldsymbol{F_s}$ **(MHz)**	$\boldsymbol{N}$	$\boldsymbol{BW}$	$\boldsymbol{\mathcal{R}}$ **(MBd)**	$\boldsymbol{\mathcal{F}}$ **(MHz)**
I	1.00	2^{16}	16	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
II	1.00	2^{16}	16	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
III	1.00	2^{16}	16	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
IV	1.00	2^{16}	16	$[0.01 : 0.01 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
V	1.00	2^{16}	16	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
VI	1.00	2^{16}	16	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
VII	10.00	2^{16}	16	$[1.00 : 1.00 : 5.00]$	$[-5.00 : 0.05 : 5.00]$
VIII	1.00	2^{12}	16	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
IX	1.00	2^{16}	8	$[0.10 : 0.10 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
X	1.00	2^{16}	16	$\{0.10\}$	$[-0.50 : 0.01 : 0.50]$
XI	1.00	2^{16}	16	$[0.0999 : 10^{-6} : 0.1001]$	$\{0.05\}$
XII	1.00	2^{16}	16	$[0.05 : 0.05 : 0.50]$	$[-0.50 : 0.05 : 0.50]$
XIII	1.00	2^{16}	16	$[0.05 : 0.05 : 0.50]$	$[-0.50 : 0.05 : 0.50]$

The plots display the requested physical candidates normalized by the sample rate: their horizontal coordinates are α_v/F_s and f_k/F_s in cycles per sample. The axis labels v and k denote these normalized candidate coordinates, and the black curves show the nonnegative test statistic $T_y(v,k) = |\widehat{C}_y(\alpha_v, f_k)|^2$. The requested CF grids include both Nyquist endpoints; because $-F_s/2$ and $+F_s/2$ represent the same discrete-time frequency, they map to the same signed control word and oscillator setting.

Experiment XI requests 201 symbol-rate values at 1-Hz intervals. At $F_s = 1$ MHz, the 16-bit control word has a frequency quantum of $F_s/2^{16} = 15.2588$ Hz, so nearest-

word quantization maps the request sweep to 14 distinct hardware settings. The plot therefore illustrates the relationship between a fine requested grid and the implemented control-word resolution.

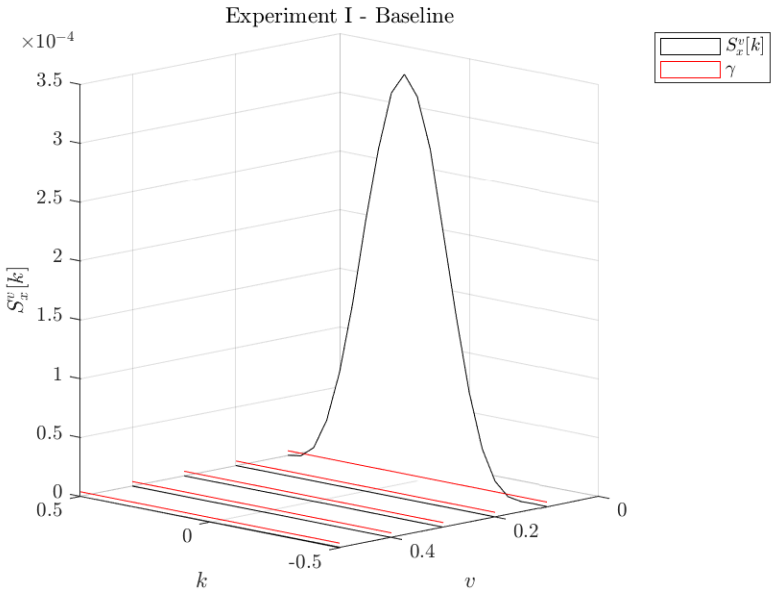

Figure 35: Streaming-SCA test-statistic plot for Experiment I.

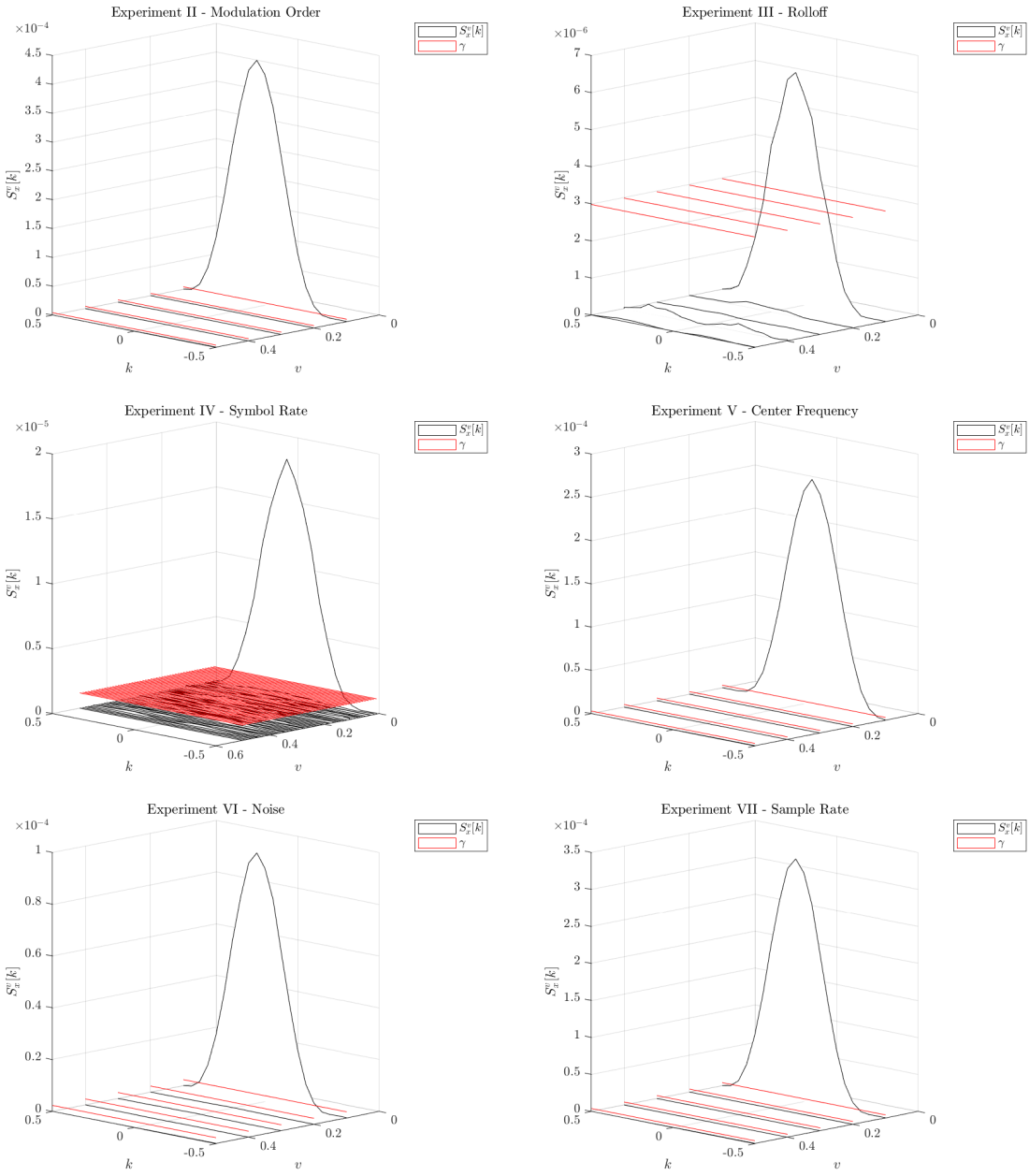

Figure 36: Streaming-SCA test-statistic plots for Experiments II–VII.

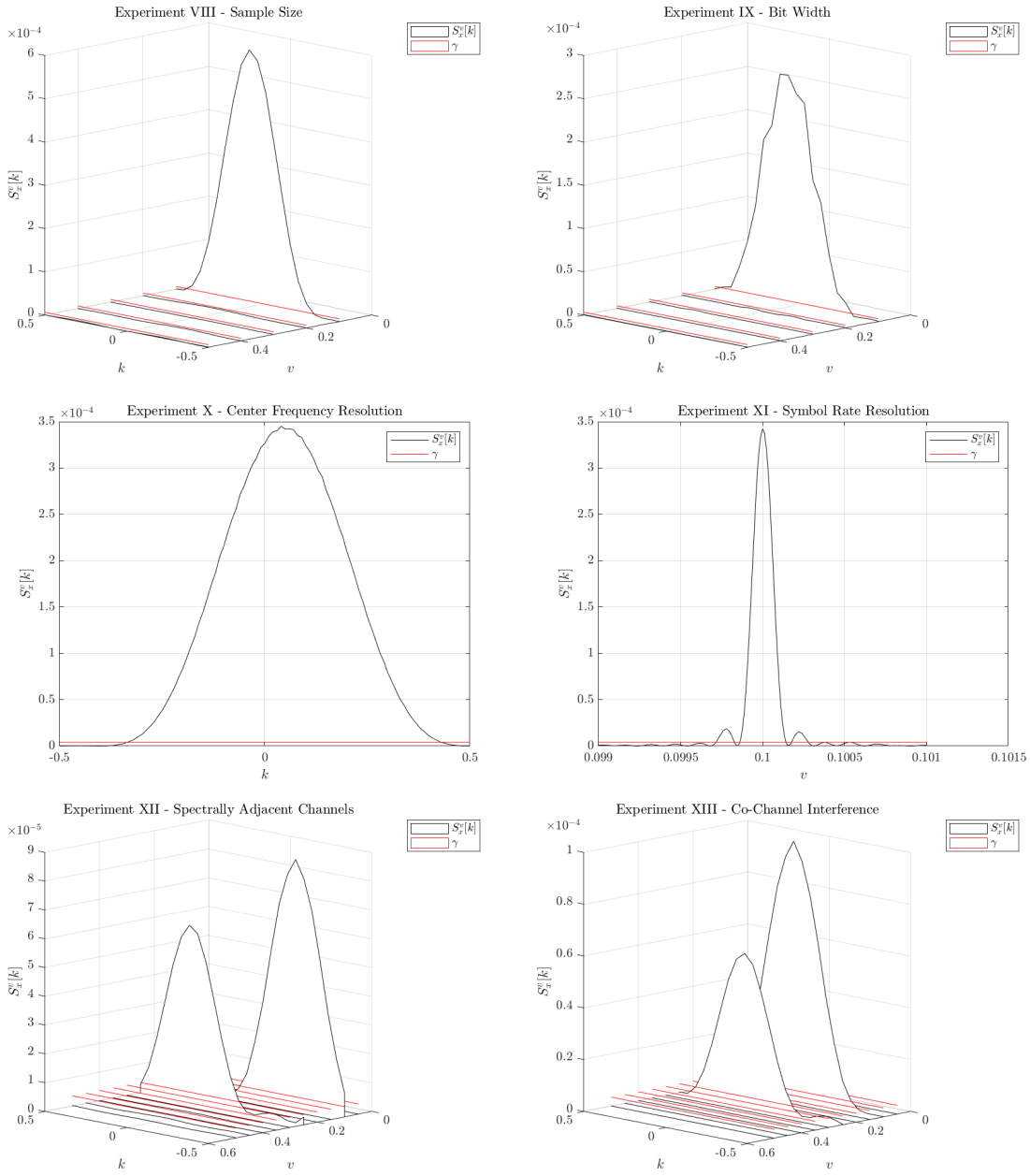

Figure 37: Streaming-SCA test-statistic plots for Experiments VIII–XIII.

4.2 Results

The simulation results are summarized in Table XIV, and Table XV reports the FPGA resource utilization. Every experiment produces at least one statistic above the selected threshold, and eleven identify the transmitted pair or pairs. Experiment IV uses the configured input rate of 0.01 MBd, and Experiment XIII identifies the second CF at 0.10 MHz, matching the experiment definitions and plotted peaks. Experiment VII selects $R_s = 1.00$ MBd for a 0.10 MBd input because 1.00 MBd is the first available receiver-grid rate. Experiment X selects $f_c = 0.04$ MHz, the nearest evaluated 10-kHz grid point to the transmitted 0.05 MHz feature. These selections demonstrate the expected relationship between the configured candidate grid and estimator resolution. The simulations use the Chapter III definitions for power format, record boundaries, accumulator sizing and normalization, and continuous-rate flow control.

The FPGA implementation flow targets a ZedBoard XC7Z020CLG484-1 with Vivado 2019.2.1 and the default synthesis and implementation strategies. Table XV presents the resulting utilization categories for each experiment. The common 16-bit configuration uses 814 LUTs, 3 LUTRAMs, 391 flip-flops, 15 BRAM resources, 16 DSP resources, and 108 I/O resources; the 8-bit configuration in Experiment IX reduces the LUT, flip-flop, and I/O counts.

At one valid sample per clock, the nominal sample-processing time of a serial sweep is $N|\mathcal{R}||\mathcal{F}|/F_s$, excluding pipeline and control overhead. The simulation times follow this serial-sweep scaling. Experiment VIII uses $N = 4096$, five SR candidates, 21 CF candidates, and a 1-MHz sample clock, giving a nominal sample-processing time of 0.430080 s. Its simulated time is 0.430241 s, with the additional 161 μs accounting for finite pipeline and control overhead. Experiment IV uses a 50-rate by 21-frequency grid and therefore has a nominal sample-processing time of 68.8128 s before pipeline and control overhead.

Table XIV: Streaming-SCA simulation results.
Here, $\gamma = \beta \widehat{P}_y^2$ with $\beta = 10^{-4}$; $\widehat{P}_y^2$ is dimensionless after input normalization and is rounded to six significant digits. Each tuple is one selected nominal (R_s, f_c) pair in (MBd, MHz). The Experiment IV time is the nominal sample-processing value before pipeline and control overhead.

Expt.	**Processing time (s)**	$\widehat{\boldsymbol{P}}_{\boldsymbol{y}}^{\boldsymbol{2}}$	**Nominal pair(s)**
I	6.881441	0.0390606	(0.10, 0.05)
II	6.881441	0.0398833	(0.10, 0.05)
III	6.881441	0.0298027	(0.10, 0.05)
IV	68.812800	0.0117707	(0.01, 0.05)
V	6.881441	0.0296905	(0.10, −0.05)
VI	6.881441	0.0230728	(0.10, 0.05)
VII	6.881441	0.0394264	(1.00, 0.05)
VIII	0.430241	0.0613674	(0.10, 0.05)
IX	6.881441	0.0323822	(0.10, 0.05)
X	6.619201	0.0390606	(0.10, 0.04)
XI	13.243154	0.0390606	(0.10, 0.05)
XII	13.762841	0.0353092	(0.10, −0.20); (0.25, 0.20)
XIII	13.762841	0.0341933	(0.10, 0.05); (0.25, 0.10)

Table XV: FPGA resource utilization by experiment for the ZedBoard XC7Z020CLG484-1 implementation.

Expt.	**LUT**	**LUTRAM**	**FF**	**BRAM**	**DSP**	**I/O**
I	814	3	391	15	16	108
II	814	3	391	15	16	108
III	814	3	391	15	16	108
IV	814	3	391	15	16	108
V	814	3	391	15	16	108
VI	814	3	391	15	16	108
VII	814	3	391	15	16	108
VIII	814	3	391	15	16	108
IX	674	3	319	15	16	100
X	814	3	391	15	16	108
XI	814	3	391	15	16	108
XII	814	3	391	15	16	108
XIII	814	3	391	15	16	108

CHAPTER V

CONCLUSION

We investigated a low-memory, FPGA-targeted VHDL architecture for M-QAM-SRRC SR–CF detection. We began by defining the streaming SCA's scope and then described its sample-by-sample datapath. The architecture uses custom selected-frequency pipelines in place of an FFT sample buffer, while MATLAB, Python, VHDL, and the Vivado toolchain provide an end-to-end modeling, simulation, synthesis, and implementation flow. Thirteen VHDL simulation experiments characterize the detector, and the resource-utilization results describe its FPGA footprint. The implementation targets a ZedBoard XC7Z020CLG484-1 with Vivado 2019.2.1. The RTL uses the fixed-point formats, record boundaries, accumulator sizing and normalization, and continuous-rate flow control defined throughout the design.

As discussed in Section 3.2, the serial processing time grows as the SR and CF candidate grids become finer. Koch first forms spectral segments from a thresholded Welch PSD [5]. After centering and dynamically low-pass filtering each segment, the cyclic-autocorrelation FFT (CAC-FFT) forms $z[n] = |x[n]|^2$ and uses an N-point FFT to evaluate the zero-lag CAC on a uniformly spaced SR grid; peak finding selects candidate rates. For each selected rate, the carrier-frequency detector sweeps that segment across frequency offsets, applies the rate-dependent low-pass filter, evaluates a single-rate CAC, and thresholds the normalized peak. The streaming hardware in this thesis implements the serial carrier sweep. A future preprocessing stage can add

the PSD segmentation and CAC-FFT candidate selection so that the sweep evaluates only the candidates selected from the observed spectrum; the resulting acceleration scales with the number of candidates passed to the streaming stage.

Decimation provides another path for reducing the sample rate processed by a candidate branch. After the band of interest is centered, an anti-alias low-pass filter precedes integer downsampling by D_{dec}, giving $F_s' = F_s/D_{\mathrm{dec}}$. The selected value of D_{dec} preserves the occupied bandwidth and the cyclic frequencies needed by the detector under the new Nyquist limit. Integrating this stage also updates the NCO control-word mapping, the observation interval represented by N samples, and the scheduling of parallel streaming-SCA instances.

The constant $\beta = 10^{-4}$ used in the characterization provides the empirical threshold operating point for the thirteen simulations. A Monte Carlo receiver-operating-characteristic campaign over the intended channels and signal classes can extend this characterization with calibrated detection and false-alarm probabilities. The row-maximum rule intentionally selects one CF for each SR candidate. A multi-signal extension can retain multiple local CF maxima above threshold and apply nonmaximum suppression or clustering. The same clustering stage can consolidate correlated above-threshold responses produced by a fine SR grid around one physical feature before reporting the waveform count.

Lastly, the characterization focuses on 4-QAM-SRRC and 16-QAM-SRRC signals. The same datapath can support additional waveform families by configuring candidate grids, filters, and thresholds for their cyclic features. Future evaluations can extend the design to rectangular pulse shapes, FSK, OOK, MSK, DSSS, DPSK, Manchester coding, and other modulations while retaining the same streaming arithmetic and control framework.

REFERENCES

[1] S. Bandyopadhyay, G. P. Subramanian, R. Foust, D. Morgan, S.-J. Chung, and F. Y. Hadaegh. A review of impending small satellite formation flying missions. In *Proceedings of the 53rd AIAA Aerospace Sciences Meeting*, pages 1–17, Kissimmee, Florida, 2015. American Institute of Aeronautics and Astronautics. doi: 10.2514/6.2015-1623.

[2] W. A. Gardner. *Introduction to Random Processes with Applications to Signals and Systems*. McGraw-Hill, New York City, New York, 2 edition, 1990.

[3] W. A. Gardner, A. Napolitano, and L. Paura. Cyclostationarity: Half a century of research. *Signal Processing*, 86(4):639–697, 2006. doi: 10.1016/j.sigpro.2005.06.016.

[4] I. Ilyas, S. Paul, A. Rahman, and R. K. Kundu. Comparative evaluation of cyclostationary detection based cognitive spectrum sensing. In *2016 IEEE 7th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 1–7, New York City, New York, 2016. IEEE. doi: 10.1109/UEMCON.2016.7777887.

[5] M. V. Koch and J. A. Downey. Interference mitigation using cyclic autocorrelation and multi-objective optimization. Technical Report NASA/TM–2019-220226, National Aeronautics and Space Administration, Glenn Research Center, Cleveland, Ohio, July 2019.

[6] Y. C. Liang. Spectrum sensing theories and methods. In *Dynamic Spectrum Management: From Cognitive Radio to Blockchain and Artificial Intelligence*, pages 41–85. Springer, Singapore, 2020. doi: 10.1007/978-981-15-0776-2_3.

[7] C. M. Spooner. *Theory and Application of Higher-Order Cyclostationarity*. PhD thesis, University of California, Davis, Davis, California, June 1992.

[8] H. L. Van Trees. Classical detection and estimation theory. In *Detection, Estimation, and Modulation Theory, Part I*, pages 19–165. Wiley-Interscience, Hoboken, NJ, 2001. doi: 10.1002/0471221082.ch2.

[9] Xilinx, Inc. Zynq-7000 SoC Data Sheet: Overview. Technical Report DS190, Xilinx, Inc., July 2018. Version 1.11.1.

APPENDIX

APPENDIX A

SOURCE CODE

The following listing is imported directly from the script used to generate Figure 16, so the appendix and the executable model remain synchronized. The model uses one receiver-noise realization for both signals, applies a common AGC scale factor, and compares $T_y(v,k) = |\widehat{C}_y(\alpha_v, f_k)|^2$ with the constant threshold $\gamma = \beta \widehat{P}_y^2$.

```
%%
% Dylan J. Gormley (NASA GRC/LCI) - 2020

clear
close all
rng(20210901,'twister')
%%
% Transmitter: M-QAM-SRRC

Nmessages = 2^16; % number of transmitted symbols per component
Fs        = 1.0e6; % sample rate

Rs        = [0.10 0.20]*1e6; % symbol rate
fc        = [0.05 0.10]*1e6; % carrier frequency
M         = [4      16];     % modulation order
Rolloff   = [1.00 0.35];     % excess bandwidth
EbN0      = [14.0  9.0];    % energy per bit over noise density in dB

% Extract clean signals. A single shared receiver noise floor is
   added
% below; independent transmitter noise realizations must not be
   stacked.
[~, syms_pb1_clean] = qam_srrc_xmtr(Nmessages, Fs, Rs(1), fc(1),
   M(1), Rolloff(1), EbN0(1));
[~, syms_pb2_clean] = qam_srrc_xmtr(Nmessages, Fs, Rs(2), fc(2),
   M(2), Rolloff(2), EbN0(2));

%%
% RX RF front end

% Sample size used to compute one point.
Ncapture = 2^16;
components = [syms_pb1_clean(1:Ncapture), syms_pb2_clean(1:Ncapture)];
```

```
% Realize both requested Eb/N0 values against one physical noise
   floor.
% Keep signal 1 at its generated level and scale signal 2
   accordingly.
SPS       = Fs./Rs;
CNR_target = EbN0 + 10*log10(log2(M)) - 10*log10(SPS);
component_pwr = mean(abs(components).^2,1);
noise_pwr    = component_pwr(1)/10^(CNR_target(1)/10);
target_pwr   = noise_pwr*10.^(CNR_target/10);
components   = components.*sqrt(target_pwr./component_pwr);

unit_noise = (randn(Ncapture,1) + 1j*randn(Ncapture,1))/sqrt(2);
noise     = unit_noise*sqrt(noise_pwr/mean(abs(unit_noise).^2));
syms_rx   = sum(components,2) + noise;

% Simulate receiver AGC. Its common scale factor preserves both
   CNRs.
norm_factor = modnorm(syms_rx,'peakpow',1);
agc        = syms_rx*norm_factor;

measured_CNR =
   10*log10(mean(abs(components).^2,1)/mean(abs(noise).^2));
measured_EbN0 = measured_CNR - 10*log10(log2(M)) + 10*log10(SPS);
assert(all(abs(measured_EbN0-EbN0) < 0.05), ...
    'A realized Eb/N0 differs from its request by at least 0.05 dB.');
fprintf('Measured Eb/N0: %.2f dB, %.2f dB\n',measured_EbN0);

%%
% Receiver: streaming SCA (fully vectorized)

% Valid symbol-rate range is (0, Fs/2].
a_step = min(Rs);
a_array = (a_step:a_step:Fs/2)';

% The half-open 10 kHz grid includes both transmitted carrier
   frequencies
% exactly without duplicating the equivalent -Fs/2 and +Fs/2
   Nyquist LOs.
fc_step = 0.010e6;
fc_array = (-Fs/2:fc_step:Fs/2-fc_step)';

% Two-tap low-pass filter
hlpf = [0.5,0.5];
```

```
n = (0:Ncapture-1)';

% 1. Calculate frequency bins once
vbin = round(a_array/Fs*Ncapture) + 1;

% 2. Generate downconverting local oscillators. The transmitter
   uses the
% positive-frequency convention exp(+j*2*pi*fc*n/Fs).
LO_matrix = exp(-1j*2*pi*n*(fc_array')/Fs);

% 3. Apply the local oscillators using implicit expansion
mixed_signal = agc(1:Ncapture) .* LO_matrix;

% 4. Channelize
channelized = filter(hlpf, 1, mixed_signal);

% 5. Symbol rate estimation
R = abs(channelized).^2;

% 6. Vectorized Goertzel algorithm
S = goertzel(R, vbin) / Ncapture;

% Test statistic and constant threshold.
T = abs(S).^2; % test statistic
beta       = 1e-3; % user-selected threshold scale
P_y_hat    = sum(abs(agc).^2)/Ncapture;
gamma      = beta*P_y_hat^2;
threshold  = gamma*ones(size(T));

% Report the strongest carrier candidate at each transmitted symbol
   rate.
peak_T     = zeros(size(Rs));
peak_fc    = zeros(size(fc));
is_detected = false(size(Rs));
for idx = 1:numel(Rs)
    row = find(a_array == Rs(idx),1);
    [peak_T(idx),column] = max(T(row,:));
    peak_fc(idx) = fc_array(column);
    is_detected(idx) = peak_T(idx) > gamma;
    fprintf(['Signal %d: Rs = %.0f kBd, estimated fc = %.0f kHz, ' ...
        'T/gamma = %.2f, detected = %s\n'],idx,Rs(idx)/1e3, ...
        peak_fc(idx)/1e3,peak_T(idx)/gamma,string(is_detected(idx)));
end
assert(all(is_detected), 'At least one transmitted signal was not
   detected.');
```

```
assert(all(peak_fc == fc), ...
    'At least one detected carrier does not match its transmitted
       carrier.');

% Plot the statistic, threshold, and the decisions using shared
    thesis style.
[fig, colors] = thesis_figure();
statistic_plot = waterfall(fc_array/1e6,a_array/1e6,T);
statistic_plot.EdgeColor = colors.blue;
statistic_plot.FaceColor = 'none';
hold on
threshold_plot = waterfall(fc_array/1e6,a_array/1e6,threshold);
threshold_plot.EdgeColor = colors.orange;
threshold_plot.FaceColor = 'none';
for idx = 1:numel(Rs)
    if is_detected(idx)
        marker_color = colors.green;
    else
        marker_color = colors.orange;
    end
    scatter3(peak_fc(idx)/1e6,Rs(idx)/1e6,peak_T(idx),50, ...
        marker_color,'filled','MarkerEdgeColor',colors.gray, ...
        'HandleVisibility','off');
end
view(225,20)
title('Streaming SCA Detection of Two M-QAM-SRRC Signals')
xlabel('$f_c$ (MHz)')
ylabel('$R_s$ (MBd)')
zlabel('$T_y(v,k)=|\widehat{C}_y(\alpha_v,f_k)|^2$')
legend([statistic_plot,threshold_plot], ...
    {'$T_y(v,k)$','$\gamma=\beta\widehat{P}_y^2$'}, ...
    'Location','northeast')
thesis_export(fig,'qam_srrc_rs');
```